

Database :-

=====

=> a db is a organized collection of interrelated data , for ex a bank db stores data related to cust,accts,trans etc and a univ db stores data related to course,students,faculty etc.

=> db is created in hardk disk and it is private memory

Types of Databases :-

=====

- 1 OLTP DB (online transaction processing)
- 2 OLAP DB (online analytical processing)

=> organizations uses OLTP for storing day-to-day transactions and OLAP for analysis.

=> OLTP is for running business and OLAP is for to analyze business

=> day-to-day operations on db includes

C create
R read
U update
D delete

DBMS :-

=>DBMS stands for db management system

=> DBMS is software used to work with db.

=> using DBMS software users can create db and work with db.

=> DBMS is an interface between users and database.

USER-----DBMS-----DB

Evolution of DBMS :-

1960	FMS (file mgmt system)
1970	HDBMS (hirerarchical dbms) NDBMS (network dbms)
1980	RDBMS (relational dbms)
1990	ORDBMS (object relational dbms)

RDBMS :-

=====

=> RDBMS concepts introduced by Edgar Frank CODD

=> E.F.CODD introduced 12 rules called codd rules

=> a db that supports all 12 rules called perfect rdbms software

information rule :-

=> according to information rule data must be organized in tables i.e. rows and columns

CUST

```

cid name  addr  => columns / fields / attributes
10 sachin  hyd
11 rahul blr
12 vijay del  => row / record / tuple

```

```

Database    => Collection of tables
Table       => Collection of rows & cols
Row         => Collection of field values
Column      => Collection of values of one field

```

=> every table must contain primary key to uniquely identify the rows

ex :- empid, accno, aadharno, panno, voterid

=> one table related to another table using foreign key

ex :-

```

projects
projid  pname  duration  cost  client
1000    AAA    3          200   Bank
1001    BBB    2          100   Airlines

```

```

emp
empid   ename  sal   job   projid (fk)
100     A      30K   SE    1000
101     B      80K   SSE   1001
102     C      120K  TL    1000

```

=> To establish relationship between two tables , take primary key of one table and add it to another table as foreign key.

RDBMS features :-
=====

- 1 easy to access and manipulate data
- 2 more security
- 3 gurantees data quality
- 4 supports data sharing
- 5 supports ACID properties

```

A => atomocity
C => consistency
I  => isolation
D  => durability

```

RDBMS softwares :- (SQL Databases)

```

sql server microsoft
oracle             oracle corp
mysql              oracle corp
postgresql         postgresql
db2                ibm

```

No-SQL Databases :-

```

MongoDB
cassandra
redis

```

SQL

No-SQL

Database	Database
Tables	Collections
Rows	Documents
Columns	Fields

emp	{	
id	name	sal
1	A	3000
2	B	4000
	{	
	id : 2,	

id : 1,

name : "A",
sal : 3000
},

name : "B",
sal : 4000
}

insert
update
delete

db.emp.insert()
db.emp.update()
db.emp.delete()

best for structured data best for unstructured data

Databases :- SQL SERVER , MongoDB

4-apr-26

ORDBMS :-

=> It is the combination of RDBMS & OOPS

ORDBMS = RDBMS + OOPS (reusability)

=> RDBMS doesn't support reusability but ORDBMS supports reusability

RDBMS :-

CUST
cid name hno street city state pin

EMP
empid ename hno street city state pin

ORDBMS :-

ADDR
hno street city state pin

CUST
cid name addr

EMP
empid ename addr

ORDBMS softwares :-

sql server
oracle
postgresql

vehicle	dbms
2 wheeler	rdbms
honda activa	sql server

DB Development Life Cycle :-

=====

- 1 Analysis
- 2 Design
- 3 Development
- 4 Testing
- 5 Deployment
- 6 Maintenance

Design :-

=====

- => Designing db means designing tables
- => DB is designed by DB Designers / Architects
- => DB is designed by using

- 1 ER Model (Entity Relationship)
- 2 Normalization

Development :-

=====

- => DB is developed by DB Developers & DBAs (DB Admin)
- => DB is developed by using any rdbms tool like sql server

Developers	DBA
creating tables	installation of sql server
creating views	creating database
creating synonyms	creating logins
creating sequences	db backup & restore
creating indexes	db export & import
creating procedures	performance tuning
creating functions	
creating triggers	
writing queries	

Testing :-

=====

- => DB is tested by QA (Quality Assurance) team by using

- 1 manually
- 2 automation

Deployment :-

=====

- => moving db from DEV server to PROD server is called deployment
- => after deploying db then end users can use db for day-to-day transactions

summary :-

what is db ?
what is dbms ?

what is rdbms ?
what is ordbms ?
what is db development ?

=====

6-apr-26

SQL SERVER :-
=====

=> sql server is a rdbms software from microsoft used to create and to manage db
=> using sql server users can create database and store,update,delete data in db.
=> sql server is used for db development and administration

versions :-

versions :-

SQL SERVER 1.1	1991
SQL SERVER 4.2	1993
SQL SERVER 6.0	1995
SQL SERVER 6.5	1996
SQL SERVER 7.0	1998
SQL SERVER 2000	2000
SQL SERVER 2005	2005
SQL SERVER 2008	2008
SQL SERVER 2012	2012
SQL SERVER 2014	2014
SQL SERVER 2016	2016
SQL SERVER 2017	2017
SQL SERVER 2019	2019
SQL SERVER 2022	2022
SQL SERVER 2025	2025

CLIENT / SERVER :-
=====

1 SERVER
2 CLIENT

SERVER :-
=====

=> server is a system where sql server is installed and running
=> inside the server sql server manages

1 DB
2 INSTANCE

=> DB is created in harddisk and acts as permanent storage
=> INSTANCE is created in ram and acts as temporary storage
=> initially data stored in instance and after saving then it is copied to db.

CLIENT :-
=====

=> client is also system from where users can

1 connects to server

- 2 submit requests to server
- 3 receives response from server

clients :-

- 1 ssms (sql server mgmt studio)
- 2 c# application
- 3 asp.net application

USER-----SSMS-----SQL SERVER-----DB

USER-----SQLPLUS-----ORACLE-----DB

USER-----MYSQLWORKBENCH-----MYSQL-----DB

SQL :-
=====

- => SQL stands for structured query language
- => a language used to communicate with sql server
- => user communicates with sql server by sending queries
- => a query is a command / instruction / question given to sql server to perform operation on db
- => sql is originally introduced by IBM and initial name of this language was "SEQUEL"
- later it is renamed to SQL.
- => sql is common to all relational db softwares

user---ssms-----sql -----sql
server-----db

user---sqlplus-----sql-----
oracle-----db

user-----mysqlworkbench-----sql-----
mysql-----db

- => based on operations over db sql is categorized into following sub-languages

DDL (Data Definition Lang)
DML (Data Manipulation Lang)
DQL/DRL (Data Query Lang / Data Retrieval Lang)
TCL (Transaction Control Lang)
DCL (Data Control Lang)

SQL

DDL	DML	DRL	TCL	DCL
create	insert		select	commit
alter	update		rollback	revoke
drop	delete		save transaction	
truncate	merge			grant

Data & Data Definition :-
=====

empid	ename	salary	data definition / metadata
100	sachin	6000	data

- => metadata data means data about data

create => to create data definition
 alter => to modify data definition
 drop => to drop data definition
 truncate => to release memory

 insert => to create data
 update => to modify data
 delete => to remove data
 merge => combination of insert, update, delete

 select => to see the data

 commit => to save changes
 rollback => to cancel changes
 save transaction => to cancel part of the changes

 grant => to give permissions
 revoke => to take back given permissions

7-apr-26

How to connect to sql server :-

=====

=> To connect to sql server open ssms and enter following details

server name :- DESKTOP-G2DM7GI
 authentication :- windows / sql server
 login :- SA (system admin)
 password :- 123

=> click connect

SERVER
 DATABASE
 TABLE
 ROWS & COLS

what are system databases ?

- 1 master
- 2 model
- 3 msdb
- 4 tempdb

Creating New Database :-

=> In object explorer select Databases => New Database

Enter Database Name :- DB730

=> click OK

=> a new database is created with following two files

- 1 Data File (.mdf master data file) => stores data
- 2 Log File (.ldf log data file) => stores operations

Name	Type	Initial Size	Autogrowth	path
DB730		DATA	8 MB	64 MB
DB730_LOG	LOG	8 MB	64 MB	

Path :-

C:\Program Files\
Microsoft SQL Server\
MSSQL17.MSSQLSERVER09\
MSSQL\DATA\

DB730.MDF
DB730_LOG.LDF

command to create new database :-

=====

=> open master database and execute the following command

syn :- CREATE DATABASE <name>

ex :- CREATE DATABASE SALESDB

Datatypes in SQL SERVER :-

=====

=> Datatype specifies

1 type of the data allowed

2 amount of memory allocated

DATATYPES

	CHAR	INTEGER	FLOAT	CURRENCY	DATE
ASCII		UNICODE	tinyint	decimal(p,s)	smallmoney
date		smallint	numeric(p,s)	money	time
char	nchar	int		datetime2	
varchar	nvarchar	bigint			
varchar(max)	nvarchar(max)	numeric(p)			

char(size) :-

=====

=> allows character data upto 8000 chars

=> recommended for fixed length char columns

ex :- NAME CHAR(10)

sachin - - - -
wasted

vijay - - - - -
wasted

=> in char datatype extra bytes are wasted , so char is not recommended for variable length fields and char is recommended for fixed length fields.

ex :- gender char(1)

M
F

state_code char(2)

AP

TG

MH

varchar(size) :-
=====

=> allows character data upto 8000 chars
=> recommended for variable length fields

ex :- NAME VARCHAR(10)

 sachin - - - -
 released

note :-

=> char/varchar allows ascii chars (256 chars) that includes a-z,A-Z,0-9,special chars
i.e. char/varchar allows alphanumeric data.

ex :- PANNO CHAR(10)
 EMAILID VARCHAR(20)
 VEHNO CHAR(10)

varchar(max) :-
=====

=> allows character data upto 2GB

ex :- TEXT VARCHAR(max)

nchar / nvarchar / nvarchar(max) :-
=====

=> allows unicode chars (65536 chars) that includes all ascii chars and also chars belongs to different languages

=> ascii char occupies 1 byte and unicode char occupies 2 bytes

INTEGER types :-
=====

=> integer types allows numbers without decimal

Types		bytes	range
tinyint		1	0 to 255
smallint	2		-2 ¹⁵ to 2 ¹⁵ -1 (-32768 to 32767)
int	4		-2 ³¹ to 2 ³¹ -1
bigint		8	-2 ⁶³ to 2 ⁶³ -1

ex :- age tinyint
 id smallint

Numeric(p) :-
=====

=> allows numbers upto 38 digits

ex :- empid NUMERIC(4)

 100
 1000
 10000 => not allowed

aadharno	NUMERIC(12)
accno	NUMERIC(15)
phone	NUMERIC(10)

Decimal(p,s) / Numeric(p,s)
=====

=> allows numbers with decimal

p => precision => total no of digits allowed => can be upto 38
s => scale => no of digits allowed after decimal => can be between 0 to p

ex :- salary DECIMAL(7,2)

```

5000
5000.56
50000.56
500000.56 => not allowed

5000.5678 => allowed => 5000.57
5000.5648 => allowed => 5000.56

savg    DECIMAL(5,2)

```

currency :-
=====

=> currency types are used for fields related to money

types	bytes	range
smallmoney	4	-2,14,748.3648 to 2,14,748.3647.
money	8	-922,337,203,685,477.5808

to

922,337,203,685,477.5807

```

ex :- salary    smallmoney
      balance    money
      fee        smallmoney

```

DATE :-
=====

DATE => allows only date
TIME => allows only time
DATETIME2 => allows both date & time

=> default date format in sql server is yyyy-mm-dd
=> default time format in sql server is hh:mi:ss
=> dates must be between 0001-01-01 through 9999-12-31.

ex :- dob DATE

2005-10-15

login TIME

9:00:00

txndt DATETIME2

2026-04-08 7:00:00

creating tables in sql server :-

=====

```
CREATE TABLE <tablename>
(
    colname datatype(size) ,
    colname datatype(size),
    -----
)
```

Rules :-

- 1 name should start with alphabet
- 2 name should not contain spaces & special chars but allows _ \$ #
- 3 name can be upto 128 chars
- 4 table can have 1024 columns
- 5 no of rows are unlimited

Ex :-

=> create table with following structure ?

```
EMP
empid  ename    job    sal    hiredate    age    dept
```

```
CREATE TABLE emp
(
    empid    SMALLINT ,
    ename    VARCHAR(10),
    job      VARCHAR(10),
    sal      SMALLMONEY,
    hiredate DATE,
    age      TINYINT,
    dept     VARCHAR(10)
)
```

=> above command is created table structure / definition / metadata that includes columns,datatype and size

SP_HELP :- (SP => stored procedure)

=====

=> command to see the structure of the table

```
SP_HELP <tablename>
```

Ex :- SP_HELP emp

```
empid smallint      2
ename varchar       10
job  varchar        10
sal  smallmoney     4
hiredate date        3
age  tinyint         1
dept varchar        10
```

INSERTING DATA INTO TABLE :-

=====

=> "INSERT" command is used to insert data into table
=> insert command creates a new row
=> we can insert

- 1 single row
- 2 multiple rows

INSERTING SINGLE ROW :-
=====

INSERT INTO <tablename> VALUES(v1,v2,v3,-----)

Ex :-

```
INSERT INTO emp VALUES(100,'sachin','clerk',4000,'2026-04-08',40,'hr')
INSERT INTO emp VALUES(101,'arvind','manager',8000,getdate(),30,'sales')
```

INSERTING MULTIPLE ROWS :-
=====

```
INSERT INTO emp
VALUES(102,'kumar','analyst',7000,'2020-02-15',25,'it') ,
      (103,'rahul','clerk',6000,'2019-10-05',35,'sales'),
      (104,'vijay','manager',9000,'2018-05-12',32,'hr')
```

INSERTING NULLS :-
=====

- => a null means blank of empty
- => it is not equal to 0 or space
- => nulls are inserted when value is missing
- => nulls can be inserted in two ways

method 1 :- (explicit)

```
INSERT INTO emp VALUES(105,'satish',null,null,'2021-12-10',28,'it')
```

method 2 :- (implicit)

```
INSERT INTO emp(empid,ename,hiredate,age,dept)
VALUES(106,'rajiv',getdate(),35,'sales')
```

=> remaining fields job,sal are filled with nulls

9-apr-26

Operators in SQL SERVER :-
=====

1 Arithmetic Operators	=>	+	-	*	/	%	
2 Relational Operators	=>	>	>=	<	<=	=	<> !=
3 Logical Operators	=>	AND	OR	NOT			
4 Special Operators	=>	BETWEEN	IN	LIKE	IS	ANY	ALL, EXISTS, PIVOT
5 Set Operators	=>	UNION	,	UNION ALL	INTERSECT	EXCEPT	

Displaying Data :-
=====

- => "SELECT" command is used to display data from table
- => we can display all rows or specific rows
- => we can display all columns or specific columns

syntax :- SELECT columns FROM tablename

sql = english
queries = sentences

clauses = words
FROM => specify tablename
SELECT => specify column names

Ex :-

=> display employee names and salaries ?

```
SELECT ename,sal FROM emp
```

=> display names,jobs and hiredates ?

```
SELECT ename,job,hiredate FROM emp
```

=> display all the data from emp table ?

```
SELECT * FROM emp
```

* => all columns

WHERE clause :-

=====

=> where clause is used to get specific row/rows from table based on a condition

=> where clause contains condition

```
SELECT columns  
FROM tablename  
WHERE condition
```

Condition :-

=> condition is a relational expression that returns TRUE/FALSE

COLNAME OP VALUE

=> OP must be any relational operator like > >= < <= = <>

=> if cond = true row is selected

=> if cond = false row is not selected

Ex :-

=> display employee details whose id = 103 ?

```
SELECT *  
FROM emp  
WHERE empid = 103
```

=> display employee details whose name = vijay ?

```
SELECT *  
FROM emp  
WHERE ename = 'vijay'
```

=> employees age > 30 ?

```
SELECT *  
FROM emp  
WHERE age > 30
```

=> employees joined after 2020 ?

```
SELECT *
FROM emp
WHERE hiredate > 2020      => ERROR
```

```
SELECT *
FROM emp
WHERE hiredate > '2020-12-31'
```

=> joined before 2020 ?

```
SELECT *
FROM emp
WHERE hiredate < '2020-01-01'
```

=> employees not working for sales dept ?

```
SELECT *
FROM emp
WHERE dept <> 'sales'
```

Compound condition :-
=====

=> multiple conditions combined with AND / OR operators

WHERE cond1	AND	cond2	result	
T	T	T		
T	F	F		
	F		T	F
F	F	F		

WHERE cond1	OR	cond2	result	
T		T	T	
	T		F	T
	F		T	T
	F	F	F	F

=> display employees whose id = 100,103,105 ?

```
SELECT *
FROM emp
WHERE empid = 100 OR empid = 103 OR empid = 105
```

=> display employees working for sales dept age > 30 ?

```
SELECT *
FROM emp
WHERE dept='sales' AND age > 30
```

=> display employees ?

- 1 working for sales dept
- 2 age > 30
- 3 earning more than 5000

```
SELECT *
FROM emp
WHERE dept='sales'
      AND
      age > 30
      AND
      sal > 5000
```

10-apr-26

=> employees working as clerk, managers ?

```
SELECT *
FROM emp
WHERE job='clerk' OR job='manager'
```

=> employees earning more than 5000 and less than 10000 ?

```
SELECT *
FROM emp
WHERE sal>5000 AND sal<10000
```

=> employees joined in 2020 ?

```
SELECT *
FROM emp
WHERE hiredate >= '2020-01-01' AND hiredate <= '2020-12-31'
```

=> display clerk or manager earning more than 5000 ?

```
SELECT *
FROM emp
WHERE job='clerk' OR job='manager' AND sal > 5000
```

above query returns clerks earning less than 5000 because sal>5000 is applied only to manager and it is not applied to clerk because operator AND has got more priority than operator OR , to group conditions use ()

```
SELECT *
FROM emp
WHERE ( job='clerk' OR job='manager' ) AND sal > 5000
```

=>

STUDENT				
sno	sname	s1	s2	s3
1	A	80	90	70
2	B	60	50	30
3	C	10	20	70
4	D	10	20	30

=> list of students who are passed ?

```
SELECT *
FROM student
WHERE s1>=35 AND s2>=35 AND s3>=35
```

=> list of students who are failed ?

```
SELECT *
FROM student
WHERE s1<35 OR s2<35 OR s3<35
```

=> list of students who are failed exactly in one subject ?

```
SELECT *
FROM student
WHERE (s1<35 AND s2>=35 AND s3>=35)
OR
(s1>=35 AND s2<35 AND s3>=35)
```

```
OR
(s1>=35 AND s2>=35 AND s3<35)
```

=> list of students who are failed exactly in two subjects ?

```
SELECT *
FROM student
WHERE (s1<35 AND s2<35 AND s3>=35)
      OR
      (S1<35 AND s2>=35 AND s3<35)
      OR
      (S1>=35 AND S2<35 AND S3<35)
```

=> list of students who are failed in all 3 subjects ?

```
SELECT *
FROM student
WHERE s1<35 AND s2<35 AND s3<35
```

IN operator :-

=====

=> use IN operator for list comparison
=> use IN operator for "=" comparison with multiple values

```
WHERE colname = v1, v2,v3,----- => INVALID
WHERE colname IN (v1,v2,v3,---)    => VALID
```

Ex :-

=> display employees whose id = 100,103,105 ?

```
SELECT *
FROM emp
WHERE empid IN (100,103,105)
```

=> employees working as clerk,manager,analyst ?

```
SELECT *
FROM emp
WHERE job IN ('clerk','manager','analyst')
```

=> employees not working for dept hr,sales ?

```
SELECT *
FROM emp
WHERE dept NOT IN ('hr','sales')
```

BETWEEN operator :-

=====

=> use between operator for range comparison

```
ex :- 5000 to 10000
      2020 to 2026
```

```
WHERE colname BETWEEN v1 AND v2 ( >=v1 AND <=v2)
```

Ex :-

=> employees earning between 2000 and 5000 ?

```
SELECT *
```



```
FROM emp
WHERE sal BETWEEN 2000 AND 5000
```

=> employees joined in 2020 ?

```
SELECT *
FROM emp
WHERE hiredate BETWEEN '2020-01-01' AND '2020-12-31'
```

=> employees not joined in 2020 ?

```
SELECT *
FROM emp
WHERE hiredate NOT BETWEEN '2020-01-01' AND '2020-12-31'
```

=> display employees

```
1 working as clerk,manager
2 earning between 5000 and 10000
3 age between 30 and 40
4 not joined in 2020
5 not working for it dept
```

```
SELECT *
FROM emp
WHERE job IN ('clerk','manager')
      AND
      sal BETWEEN 5000 AND 10000
      AND
      age BETWEEN 30 AND 40
      AND
      hiredate NOT BETWEEN '2020-01-01' AND '2020-12-31'
      AND
      dept <> 'it'
```

=> display samsung,redmi,realme mobile phones price between 10000 and 20000 ?

```
PRODUCTS
prodid  pname price category    brand

SELECT *
FROM products
WHERE brand IN ('samsung','redmi','realme')
      AND
      price BETWEEN 10000 AND 20000
      AND
      category='mobiles'
```

11-apr-26

LIKE operator :-
=====

=> use LIKE operator for pattern comparison

```
ex :-  name starts with 's'
        name ends with 'd'
        name contains 'a'
```

```
WHERE colname LIKE 'pattern'
```

=> pattern may contain alphabets,digits,special chars and wildcard chars

wildcard chars :-

=====

% 0 or many chars

_ exactly 1 char

Ex :-

=> employees name starts with 's' ?

```
SELECT *
FROM emp
WHERE ename LIKE 's%'
```

=> employees name ends with 'd' ?

```
SELECT *
FROM emp
WHERE ename LIKE '%d'
```

=> name contains 'a' ?

```
SELECT *
FROM emp
WHERE ename LIKE '%a%'
```

=> employees name starts with vowels ?

```
SELECT *
FROM emp
WHERE ename LIKE '[aeiou]%'
```

=> employees where 'a' is the 4th char in their name ?

```
SELECT *
FROM emp
WHERE ename LIKE '___a%'
```

=> employees where 'a' is the 4th char from last ?

```
SELECT *
FROM emp
WHERE ename LIKE '%a___'
```

=> name contains 5 chars ?

```
SELECT *
FROM emp
WHERE ename LIKE '_____'
```

=> employees joined in apr month ? yyyy-mm-dd

```
SELECT *
FROM emp
WHERE hiredate LIKE '_____04____'

LIKE '%-04-%'
```

=> employees joined in 2020 year ?

```
SELECT *
FROM emp
WHERE hiredate LIKE '2020%'
```

=> employees joined in 1st 9 days of any month any year ? yyyy-mm-dd

```
SELECT *
FROM emp
WHERE hiredate LIKE '%0_'
```

IS operator :-

=====

=> use IS operator for NULL comparison

```
WHERE colname IS NULL
WHERE colname IS NOT NULL
```

=> employees not earning salary ?

```
SELECT *
FROM emp
WHERE sal IS NULL
```

=> employees earning salary ?

```
SELECT *
FROM emp
WHERE sal IS NOT NULL
```

summary :-

=====

```
WHERE COLNAME IN (V1,V2,V3,---)
WHERE COLNAME BETWEEN V1 AND V2
WHERE COLNAME LIKE 'PATTERN'
WHERE COLNAME IS NULL
```

ALIAS :-

=====

=> alias means another name or alternative name

=> use alias to change column heading

```
COLNAME /  EXPR    [AS]  ALIAS
```

Ex :-

=> display ENAME ANNUAL SAL ?

```
SELECT ename,sal*12 as annsal
FROM emp
```

```
SELECT ename,sal*12 as [annual salary]
FROM emp
```

=> display ENAME SAL HRA DA TAX TOTSAL ?

```
HRA = house rent allowance = 20% on sal
DA  = dearness allowance   = 30% on sal
TAX = 10% on sal
TOTSAL = SAL + HRA + DA - TAX
```

```
SELECT ENAME,SAL,
       SAL*0.2 AS HRA,
       SAL*0.3 AS DA,
```

```

                SAL*0.1 AS TAX,
                SAL+(SAL*0.2)+(SAL*0.3)-(SAL*0.1) AS TOTSAL
FROM EMP

SACHIN          4000   800   1200   400   5600

```

13-apr-26

ORDER BY clause :-
=====

=> order by clause is used to sort table data
=> sorting is based on one or more fields
=> we can sort either in ASC or DESC order
=> default sort order is ASC

```

SELECT columns
FROM tablename
[WHERE cond]
ORDER BY col1 ASC / DESC , col2 ASC/DESC ,-----

```

	ASC	DESC
numeric	0-9	9-0
char	a-z	z-a
date	old - new	new - old

Ex :-

=> arrange employee list name wise ascending ?

```

SELECT *
FROM emp
ORDER BY ename ASC

```

=> arrange employee list ?
display highest paid employee row first ?

```

SELECT *
FROM emp
ORDER BY sal DESC

```

1	A	3000		2	B	6000	
2	B	6000	=====>	3	C	4000	
3	C	4000		1	A	3000	

=> arrange employee list ?
employee who joined first display first ?

```

SELECT *
FROM emp
ORDER BY hiredate ASC

```

=> arrange employee list ?
dept wise asc and with in dept sal wise desc ?

```

SELECT empno,ename,sal,deptno
FROM emp
ORDER BY deptno ASC , SAL DESC

```

1	A	3000	20		6	F	7000	10	
2	B	5000	10						
3	C	4000	30	=====>	4	D	6000	20	
					2	B	5000	10	

4	D	6000	20
5	E	5000	30
6	F	7000	10

1	A	3000	20
5	E	5000	30
3	C	40000	30

=> arrange employee list ?
in each dept arrange most experienced employee first ?

```
SELECT empno,ename,hiredate,deptno
FROM emp
ORDER BY deptno ASC, hiredate ASC
```

```
7782 CLARK 1981-06-09 10
7839 KING 1981-11-17 10
7934 MILLER 1982-01-23 10
```

```
7369 SMITH 1980-12-17 20
7566 JONES 1981-04-02 20
7902 FORD 1981-12-03 20
```

=> arrange student list avg wise desc ?
if avg same then arrange m desc ?
if m also same then arrange p desc ?

```
student
sno  sname m    p    c
1    A    80    90    70
2    B    60    70    50
3    C    90    70    80
4    D    90    80    70
```

```
SELECT * , (M+P+C)/3 AS AVG
FROM STUDENT
ORDER BY (M+P+C)/3 DESC , M DESC,P DESC
```

```
4    D    90    80    70    80
3    C    90    70    80    80
1    A    80    90    70    80
2    B    60    70    50    60
```

=> display employees working as clerk,manager ?
arrange output sal wise desc order ?

```
SELECT *
FROM emp
WHERE job IN ('CLERK','MANAGER')
ORDER BY sal DESC
```

execution :-

```
FROM
WHERE
SELECT
ORDER BY
```

14-apr-26

DISTINCT clause :-
=====

=> distinct clause eliminates duplicates from select query output

```
SELECT DISTINCT col1,col2,----
```

```
FROM tablename
[WHERE cond]
[ORDER BY ---]
```

Ex :-

```
SELECT DISTINCT job FROM emp
```

```
ANALYST
CLERK
MANAGER
PRESIDENT
SALESMAN
```

```
SELECT DISTINCT deptno FROM emp
```

```
10
20
30
```

TOP clause :-
=====

=> top clause returns top n rows

```
SELECT [DISTINCT] TOP <n> colnames / *
FROM tablename
[WHERE cond]
[ORDER BY --]
```

Ex :-

=> display first 5 rows from emp table ?

```
SELECT TOP 5 * FROM emp
```

=> display top 3 highest paid employees ?

```
SELECT TOP 3 *
FROM emp
ORDER BY sal DESC
```

=> display top 3 max salaries ?

```
SELECT DISTINCT TOP 3 sal
FROM emp
ORDER BY sal DESC
```

=> display top 3 employees based on experience ?

```
SELECT TOP 3 *
FROM emp
ORDER BY hiredate ASC
```

summary :-
=====

WHERE	=> used for data filterting
ORDER BY	=> used for sorting
DISTINCT	=> used to eliminate duplicates
TOP	=> used to display top n rows

DML commands :- (Data Manipulation Lang)
=====

INSERT
UPDATE
DELETE
MERGE

TABLE = COLUMNS(structure) + ROWS (data)

DDL

DML

=> all DML commands acts on table data
=> all DML commands are auto committed (saved)
=> to stop auto commit then execute the following command

SET IMPLICIT_TRANSACTIONS ON

=> after executing above command auto commint is off
=> to save the operation execute COMMIT
=> to cancel the operation execute ROLLBACK

UPDATE command :-
=====

=> command used to modify table data
=> we can update all rows or specific rows
=> we can update single column or multiple columns

UPDATE tablename
SET colname = value , colname = value, -----
[WHERE cond]

=> update all employees comm with 500 ?

UPDATE emp SET comm = 500

=> update employee comm with 500 whose empno = 7844 ?

UPDATE emp SET comm = 500 WHERE empno = 7844

=> update employees comm with 500 whose comm = null ?

UPDATE emp SET comm = 500 WHERE comm = NULL

=> update comm with null whose comm <> null ?

UPDATE emp SET comm = NULL WHERE comm IS NOT NULL

NULL assignment =
NULL comparison IS

=> update sal with 2000 and comm with 500 whose empno = 7369 ?

UPDATE emp
SET sal = 2000 , comm = 500
WHERE empno = 7369

=> increment sal by 20% and comm by 10% those working as salesman
and joined in 1981 year ?

UPDATE emp
SET sal = sal + (sal*0.2) , comm = comm + (comm * 0.1)
WHERE job='salesman'
AND
hiredate LIKE '1981%'

=> transfer all the employees from 10th dept to 20th dept ?

```
UPDATE emp SET deptno = 20 WHERE deptno = 10
```

DELETE command :-

=====

=> command used to delete row/rows from table

=> we can delete all rows or specific rows

```
DELETE FROM tabname [WHERE cond]
```

Ex :-

=> delete all rows from emp table ?

```
DELETE FROM emp
```

=> delete employees who are joined in 1980 year ?

```
DELETE FROM emp WHERE hiredate LIKE '1980%'
```

=> delete employees whose empno = 7369,7499 ?

```
DELETE FROM emp WHERE empno IN (7369,7499)
```

15-apr-26

DDL commands :- (Data Definition Lang)

=====

CREATE

ALTER

DROP

TRUNCATE

=> DDL commands acts on table structure (columns,datatype and size)

=>DDL commands are also auto committed (saved)

=> to stop auto commit execute SET IMPLICIT_TRANSACTIONS ON

ALTER command :-

=====

=> command used to modify table structure

=> modifying structure means

- 1 adding columns
- 2 dropping columns
- 3 modifying column
 - changing datatype
 - changing size

Adding columns :-

=====

```
ALTER TABLE <tablename>
```

```
ADD colname datatype(size), colname datatype(size) ,-----
```

Ex :-

=> add column gender to emp table ?

```
ALTER TABLE emp
```



```
ADD gender CHAR(1)
```

after adding by default the column is filled with NULLs , to insert data into the new column use update command

```
UPDATE emp SET gender = 'M' WHERE empno = 7369
```

Dropping columns :-

=====

```
ALTER TABLE <tablename>
DROP COLUMN col1,col2,-----
```

Ex :-

=> drop column gender from emp table ?

```
ALTER TABLE emp
DROP COLUMN gender
```

Modifying column :-

=====

- 1 changing datatype
- 2 changing size

```
ALTER TABLE <tablename>
ALTER COLUMN colname datatype(size)
```

Ex :-

=> modify column empno datatype to int ?

```
ALTER TABLE emp
ALTER COLUMN empno INT
```

```
ALTER TABLE emp
ALTER COLUMN empno TINYINT => ERROR => existing values in empno
are out of tinyint range
```

=> increase size of ename to 20 ?

```
ALTER TABLE emp
ALTER COLUMN ename VARCHAR(20)
```

```
ALTER TABLE emp
ALTER COLUMN ename VARCHAR(5) => ERROR => some names
contains more than
5 chars
```

DROP command :-

=====

=> command used to drop table from db
=> drops table structure along with data.

```
DROP TABLE tablename
```

Ex :-

DROP TABLE emp

TRUNCATE command :-

=====

=> command delete all rows from table
=> will empty the table
=> releases memory allocated for table

TRUNCATE TABLE <tablename>

Ex :- TRUNCATE TABLE student

=> sql server goes to memory and releases all the pages allocated for table and when pages are released then data stored in pages also deleted.

differentiate between DROP,DELETE and TRUNCATE ?

DROP	DELETE / TRUNCATE
drops table structure with data	deletes only data but not structure

	DELETE	TRUNCATE
1	DML	DDL
2	can delete all rows and specific rows	can delete only all rows but cannot delete specific rows
3	where cond can be used with delete	where cond cannot be used with truncate
4	deletes row-by-row	deletes all rows at a time
5	slower	faster
6	will not release memory	releases memory
7	will not reset identity	will reset identity

SP_RENAME :- (SP => stored procedure)

=====

=> used to change table name and column name

SP_RENAME 'old name' , 'new name'

Ex :-

=> rename table student to stud ?

SP_RENAME 'student','stud'

=> rename column M to MATHS in stud table ?

SP_RENAME 'STUD.M','MATHS'

16-apr-26

Built-in Functions :-

=====

=> functions provided by sql server are called built-in function
=> a function accepts some input performs some calculation and returns one value

purpose :-
=====

- 1 Data cleaning
- 2 Data processing
- 3 Data transformation
- 4 Data Analysis

Types of functions :-

- 1 single row functions
 - character
 - date
 - numeric
 - conversion
 - special
 - analytical
- 2 multi row / aggregate functions

single row functions :-
=====

=> these functions process one row at a time and returns one value per row

character functions :-
=====

UPPER() :-
=====

=> converts string to uppercase

UPPER(arg)

arg	=>	string	'hello'
		colname	ename

Ex :-

SELECT UPPER('hello') => HELLO

LOWER() :-
=====

=> converts string to lowercase

LOWER(arg)

Ex :-

SELECT LOWER('HELLO') => hello

=> display EMPNO ENAME SAL ?
display names in uppercase ?

SELECT empno,UPPER(ename) as ename,sal FROM emp

=> convert names to uppercase in table ?

```
UPDATE emp SET ename = UPPER(ename)
```

LEN() :-
=====

=> returns string length i.e. no of chars

```
LEN(arg)
```

Ex :-

```
SELECT LEN('HELLO WELCOME')      =>    13
```

=> display employees name contains 4 chars ?

```
WHERE ename LIKE '____'
```

```
SELECT * FROM emp WHERE LEN(ename) = 4
```

=> name contains more than 5 chars ?

```
SELECT * FROM emp WHERE LEN(ename) > 5
```

LEFT() & RIGHT() :-
=====

```
LEFT(string,no of chars)  => returns chars starting from left
RIGHT(string,no of chars) => returns chars starting from right
```

Ex :-

```
SELECT LEFT('HELLO WELCOME',5)      =>    HELLO
SELECT RIGHT('HELLO WELCOME',7)     =>    WELCOME
```

=> employees name starts with 's' ?

```
WHERE ename LIKE 's%'
```

```
SELECT * FROM emp WHERE LEFT(ename,1) = 'S'
```

=> employees name ends with 's' ?

```
WHERE ename LIKE '%s'
```

```
SELECT * FROM emp WHERE RIGHT(ename,1) = 'S'
```

=> employees name starts and ends with same char ?

```
WHERE ename LIKE 'A%A'
      OR
      ename LIKE 'B%B'
      OR
      ename LIKE 'C%C'
```

```
SELECT * FROM emp WHERE LEFT(ename,1) = RIGHT(ename,1)
```

=> generate emailids for employees as follows ?

empno	ename	emailid
7369	smith	smi736@tcs.com
7499	allen	all748@tcs.com

```

SELECT empno,ename,
       LOWER(LEFT(ename,3)) + LEFT(empno,3) + '@tcs.com' as emailid
FROM emp

```

=> store emailids in db ?

step 1 :- add emailid column to emp table

```

ALTER TABLE emp
ADD emailid VARCHAR(20)

```

step 2 :- update the column with emailids

```

UPDATE emp
SET emailid = LOWER(LEFT(ename,3)) + LEFT(empno,3) + '@tcs.com'

```

SUBSTRING() :-
=====

=> returns characters starting from specific position

SUBSTRING(string,start, [no of chars])

```

SELECT SUBSTRING('HELLO WELCOME',10,3)  => COM
SELECT SUBSTRING('HELLO WELCOME',7,4)    => WELC
SELECT SUBSTRING('HELLO WELCOME',7)      => WELCOME

```

17-apr-26

CHARINDEX() :-
=====

=> returns position of a character in a string

CHARINDEX(char , string , [start])

Ex :-

```

SELECT CHARINDEX('O','HELLO WELCOME')    => 5
SELECT CHARINDEX('K','HELLO WELCOME')    => 0
SELECT CHARINDEX('O','HELLO WELCOME',6)  => 11

```

REPLICATE() :-
=====

=> repeats given char for given no of times

REPLICATE(char,length)

Ex :-

```

SELECT REPLICATE('*',5)                  => *****

```

=> display ENAME SAL ?

```

***
SELECT ename,REPLICATE('*',LEN(sal)) as sal FROM emp

```

Ex :-

```

ACCOUNTS
accno      bal
123456789518 20000

```

your a/c no XXXX9518 debited ----- ?

REPLICATE('X',4) + RIGHT(accno,4)

Ex :-

```
CREDITCARDS
card_no      name  expiry_date
1234567890126783  xxx  xxxx

12xxxx783
```

LEFT(card_no,2) + REPLICATE('x',4) + RIGHT(card_no,3)

REPLACE() :-

=====

=> function used to replace one string with another string

REPLACE(str1,str2,str3)

Ex :-

```
SELECT REPLACE('HELLO','ELL','ABC')    =>  HABCO
SELECT REPLACE('HELLO','L','ABC')      =>  HEABCABCO
SELECT REPLACE('HELLO','ELO','ABC')    =>  HELLO
```

```
SELECTT  REPLACE( '@@HE@@LL@@@','@','')    =>  HELLO
```

=> replace 1981 with 2021 in hiredate column in emp table ?

```
UPDATE EMP
SET HIREDATE = REPLACE(HIREDATE,'1981','2021')
```

=> replace april with may in hiredate column ?

```
UPDATE EMP
SET HIREDATE = REPLACE(HIREDATE , '-04-','-05-')
```

STUFF() :-

=====

=> used to replace one string with another string

=> replacing is based on start and length

STUFF(string1, start, length,string2)

Ex :-

```
1  SELECT STUFF('HELLO WELCOME',10,4,'GO')    =>  HELLO WELGO
```

```
2  CREDITCARDS
    card_no      name  expiry_date
    1234567890126783  xxx  xxxx
```

output :- 12xxxx783

STUFF(card_no , 3,11,'xxxx')

TRANSLATE() :-

=====

=> used to translate one char to another char

TRANSLATE(str1,str2,str3)

Ex :-

SELECT TRANSLATE('HELLO','ELO','ABC') => HABBC

E	=>	A
L	=>	B
O	=>	C

SELECT TRANSLATE('HELLO','L','ABC') => ERROR

=> translate function can be used to encrypt data i.e. converting plain text to cipher text

```
SELECT ename,
       TRANSLATE(sal , '0123456789.' , '$bT*@!#g%^&') as sal
FROM emp

JONES  2975      T^g!
```

=> remove all special chars from @#HE\$%LL^&O*! ?

```
SELECT
REPLACE(TRANSLATE('@#HE$%LL^&O*!','@#$$%^&*!','*****'),'*','')
```

***HE**LL**O**

REVERSE() :-

=> reverse the string

REVERSE(string)

SELECT REVERSE('HELLO') => OLLEH

=> display employees whose name is palindrome ?

SELECT * FROM emp WHERE ename = REVERSE(ename)

DATE functions :-

=====

GETDATE() :-

=====

=> returns current date

SELECT GETDATE() => 2026-04-17 08:42:46.660

18-apr-26

DATEPART() :-

=====

=> used to extractt part of the date

DATEPART(interval , date)

Ex :-

```
SELECT DATEPART(YY, GETDATE())      => 2026
      MM                          => 04
      DD                          => 17 (1-31)
                                DW => 6 (1-7)
      QQ                          => 2 (1-4)
      HH                          => 8
      MI                          => 48
      SS                          => 20
```

=> employees joined in 1980,1983,1985 ?

```
SELECT *
FROM emp
WHERE DATEPART(YY,HIREDATE) IN (1980,1983,1985)
```

=> employees joined in leap year ?

```
SELECT *
FROM emp
WHERE DATEPART(yy,hiredate)%4=0
```

=> employees joined in jan,apr,dec months ?

```
SELECT *
FROM emp
WHERE DATEPART(mm,hiredate) IN (1,4,12)
```

=> employees joined on sunday ?

```
SELECT *
FROM emp
WHERE DATEPART(dw,hiredate) = 1
```

=> employees joined in 2nd quarter of 2021 year ?

```
SELECT *
FROM emp
WHERE DATEPART(yy,hiredate) = 2021 AND DATEPART(qq,hiredate) = 2
```

DATENAME() :-
=====

=> this function is also used to get part of the date

```
DATENAME(interval , date)

      MM      DW
DATEPART      04      7
```

```
DATENAME      April      Saturday
```

=> display ENAME DAY_OF_WEEK ?

```
SELECT ename , DATENAME(DW,hiredate) as day FROM emp
```

=> waq to display on which day india got independence ?

```
SELECT DATENAME(DW,'1947-08-15') => Friday
```


FORMAT() :-
=====

=> function used to display dates in different formats

FORMAT (DATE , 'FORMAT')

Ex :-

```
SELECT FORMAT(GETDATE() , 'dd-MM-yyyy')           => 18-04-2026
SELECT FORMAT(GETDATE() , 'dd-MMM-yyyy')           => 18-Apr-2026
SELECT FORMAT(GETDATE() , 'dd-MMMM-yyyy')          => 18-April-2026
SELECT FORMAT(GETDATE() , 'dd-MM-yyyy ddd')        => 18-04-2026 Sat
SELECT FORMAT(GETDATE() , 'dd-MM-yyyy dddd')       => 18-04-2026 Saturday
SELECT FORMAT(GETDATE() , 'dd-MM-yyyy hh:mm:ss')   => 18-04-2026 08:15:57
```

=> display ENAME HIREDATE ?
display hiredates in dd-mm-yyyy format ?

```
SELECT ename,FORMAT(hiredate , 'dd-MM-yyyy') as hiredate
FROM emp
```

=> display employees joined today ?

```
INSERT INTO emp(empno,ename,sal,hiredate)
```

```
VALUES(999,'ABC',3000,getdate())
```

```
SELECT *
FROM emp
WHERE hiredate = GETDATE()           => no rows

2026-04-18    =    2026-04-18 8:25:20.123
```

```
SELECT *
FROM emp
WHERE hiredate = FORMAT(GETDATE() , 'yyyy-MM-dd')

2026-04-18 = 2026-04-18
```

DATEADD() :-
=====

=> function used to add / subtract years / months / days to / from a date

DATEADD(interval , int , date)

Ex :-

```
SELECT DATEADD( dd , 10, getdate())           => 2026-04-28
SELECT DATEADD( mm , 2, getdate())           => 2026-06-18
SELECT DATEADD( dd , -10, getdate())         => 2026-04-08
```

=> list of employees joined in last 5 years ?

```
SELECT *
FROM emp
WHERE hiredate BETWEEN DATEADD(yy,-5,getdate()) AND getdate()

WHERE hiredate >= DATEADD(yy,-5,getdate())
```

2021-04-18

=> list of employees joined in last 3 months ?

```
WHERE hiredate >= DATEADD(mm,-3,getdate())
```

=> employees joined in last 15 days ?

```
WHERE hiredate >= DATEADD(dd,-15,getdate())
```

Ex :-

GOLD_RATES

dateid	rate
2020-01-01	?
2020-01-02	?
2020-01-03	?

2026-04-18	?
------------	---

=> display today's gold rate ?

=> display yesterday's gold rate ?

=> display last month same day gold rate ?

=> display last year same day gold rate ?

=> display last 1 month gold rates ?

20-apr-26

DATEDIFF() :-

=====

=> returns difference between two dates in given interval

```
DATEDIFF( interval , start date, end date)
```

Ex :-

SELECT DATEDIFF(DD, '2025-04-20', GETDATE())	=>	365
SELECT DATEDIFF(MM, '2025-04-20', GETDATE())	=>	12
SELECT DATEDIFF(YY, '2025-04-20', GETDATE())	=>	1

=> display ENAME EXPERIENCE in years ?

```
SELECT  ename ,
        DATEDIFF(yy,hiredate ,getdate() ) as experience
FROM emp
```

SMITH	46
ALLEN	45

=> display ENAME EXPERIENCE ?

M years N months

experience = 40 months = 3 years 4 months

years = months / 12 = 40 / 12 = 3

months = months % 12 = 40%12 = 4

```
SELECT  ename ,
        DATEDIFF(MM,hiredate,getdate()) / 12 as years ,
        DATEDIFF(MM,hiredate,getdate())%12 as months
FROM emp
```

SMITH	45	4
-------	----	---

ALLEN 45 2

=> display employees having more than 45 years of experience ?

```
SELECT *
FROM emp
WHERE DATEDIFF(YY,hiredate,GETDATE()) > 45
```

EOMONTH() :-

=====

=> returns end of month i.e. last day of the month

EOMONTH(DATE,[INT])

Ex :-

```
SELECT EOMONTH(GETDATE())    =>    2026-04-30
SELECT EOMONTH(GETDATE() , -1) =>    2026-03-31
SELECT EOMONTH(GETDATE() , 1)  =>    2026-05-31
```

=> display current month 1st day ?

=> display next month 1st day ?

=> display current year 1st day ?

=> display next year 1st day ?

Numeric Functions :-

=====

ABS() :-

=> returns absolute value i.e. positive value

```
SELECT ABS(-10)    => 10
```

POWER() :-

=> calculates power

```
SELECT POWER(3,2)    =>    9
```

SQRT() :-

=> calculate square root

```
SELECT SQRT(16) =>    4
```

SQUARE() :-

=> calculates square of a given number

```
SELECT SQUARE(5)    =>    25
```

Rounding Numbers :-

ROUND
CEILING
FLOOR

ROUND() :-

=> rounds number to integer or to decimal places
=> round function acts according to average

ROUND(number, decimal places)

number >= avg => rounded to highest
number < avg => rounded to lowest

Ex :-

SELECT ROUND(38.45678,0) => 38

38-----38.5-----39

SELECT ROUND(38.55678,0) => 39

SELECT ROUND(38.55678,1) => 38.6

SELECT ROUND(38.5638,2) => 38.56

SELECT ROUND(38.5678,3) => 38.568

SELECT ROUND(384, -1) => 380

380-----385-----390

SELECT ROUND(384, -2) => 400

300-----350-----400

SELECT ROUND(384, -3) => 0

0-----500-----1000

=> SELECT ROUND(4567, -1) , ROUND(4567, -2) , ROUND(4567, -3) ?

o/p :- 4570 4600 5000

=> round salaries to hundreds in emp table ?

UPDATE emp SET sal = ROUND(sal , -2)

=> SELECT ROUND (ROUND(387, -1), -2) ?

o/p :- 400

ROUND(387, -1) => 390

SELECT ROUND(390, -2) => 400

CEILING() :-

=====

=> rounds number always to highest

CEILING(number)

Ex :-

```
SELECT CEILING(3.1)    =>    4
```

FLOOR() :-

=> rounds number always to lowest

FLOOR(number)

Ex :-

```
SELECT FLOOR(3.9)      =>    3
```

21-apr-26

Conversion Functions :-

=====

=> conversion means converting one datatype to another datatype

- 1 CAST
- 2 CONVERT

CAST :-

CAST(source-expr as target-type)

Ex :-

```
SELECT CAST(10.5 AS INT)      =>    10
```

=> display smith earns 800
allean earns 1600 ?

```
SELECT ename + ' earns ' + sal FROM emp => ERROR
```

in concatenation all expressions must be char type

```
SELECT ename + ' earns ' + CAST(sal AS VARCHAR) FROM emp
```

=> display smith joined on 17-DEC-1980 as clerk ?

```
SELECT ename + ' joined on ' +  
          FORMAT(hiredate, 'dd-MMM-yyyy') + ' as ' +  
          job  
FROM emp
```

CONVERT() :-

CONVERT(target-type , source-expr)

Ex :-

```
SELECT CONVERT( INT, 10.5)    =>    10
```

difference between CAST & CONVERT ?

=> using convert we can display dates in different formats but not possible using cast

Displaying Dates in different formats :-

CONVERT(VARCHAR , DATE, style-number)

Ex :-

SELECT CONVERT(VARCHAR,GETDATE(),101) => 04/21/2026

SELECT CONVERT(VARCHAR,GETDATE(),105) => 21-04-2026

SELECT CONVERT(VARCHAR,GETDATE(),110) => 04-21-2026

SELECT CONVERT(VARCHAR,GETDATE(),115) => 080319

what are the functions to display dates in different formats ?

- 1 FORMAT
- 2 CONVERT

difference between format & convert ?

in format function format is user defined but in convert function format is predefined.

		format , convert	
yyyy-mm-dd	=====>		mm/dd/yyyy
		dd-mm-yyyy	
		convert	
dd-mm-yyyy	=====>		yyyy-mm-dd

Ex :-

=> convert '21-04-2025' to date ?

SELECT CONVERT(DATE , '21-04-2025',105) => 2025-04-21

=> calculate the difference between '04/21/2025' and getdate() in years ?

SELECT DATEDIFF(YY, CONVERT(DATE, '04/21/2025',101),GETDATE()) => 1

SELECT DATEDIFF(YY, CONVERT(DATE , '21-04-2025',105),GETDATE()) => 1

Special functions :-

=====

How to handle nulls in sql server ?

- 1 ISNULL function
- 2 COALESCE function

ISNULL function :-

=====

=> function used to convert null values

ISNULL(arg1,arg2)

if arg1 = null returns arg2

if arg1 <> null returns arg1 only

=> both arguments must be same type

Ex :-

SELECT ISNULL(200,300) => 200

SELECTT ISNULL(NULL,300) => 300

=> display ENAME SAL COMM TOTSAL ?

TOTSAL = SAL + COMM

SELECT ename, sal ,comm , sal + comm as totals FROM emp

SMITH	800.00	NULL	NULL
ALLEN	1600.00	300.00	1900.00

SELECT ename,sal,comm, sal+ISNULL(comm,0) as totals FROM emp

SMITH	800.00	NULL	800.00
ALLEN	1600.00	300.00	1900.00

=> display ENAME SAL COMM ?

if comm = NULL display N/A ?

SELECT ename,sal,ISNULL(comm,'n/a') as comm FROM emp => ERROR

SELECT ename,sal,ISNULL(CAST(comm AS VARCHAR),'N/A') as comm
FROM emp

SMITH	800.00	N/A
ALLEN	1600.00	300.00

22-apr-26

=> display SNO M P C TOTAL AVG ?
if marks = null display ABSENT ?

student

sno	sname	m	p	c
1	A	80	90	70
2	B	NULL	60	70
3	C	70	NULL	80
4	D	50	60	NULL

SELECT sno,
ISNULL(CAST(m AS VARCHAR), 'ABSENT') as m ,
ISNULL(CAST(p AS VARCHAR), 'ABSENT') as p,
ISNULL(CAST(c AS VARCHAR), 'ABSENT') as c,
ISNULL(m,0) + ISNULL(p,0) + ISNULL(c,0) as total,
(ISNULL(m,0) + ISNULL(p,0) + ISNULL(c,0))/3 as avg
FROM student

COALESCE() :-
=====

=>function returns first not null expression

COALESCE(arg1,arg2,arg3,-----)

Ex :-

1 SELECT COALESCE(100,200,300) => 100

2 SELECT COALESCE(NULL,200,NULL,300) => 200

=> display ENAME SAL COMM TOTSAL ?

```
SELECT  ename,sal,comm, sal + COALESCE(comm,0)  as totsals FROM emp
```

=>

```
CUST
cid      cname addr1 addr2 addr3
1   A      HYD   BLR   --
2   B      ---   MUM   ---
3   C      --    ---   DEL
```

=> display CID CNAME ADDR ?

```
SELECT cid,cname,
        COALESCE(addr1,addr2,addr3) as addr
FROM cust
```

```
1  A      HYD
2  B      MUM
3  C      DEL
```

Analytical Functions / Window Functions :-

=====

=> these functions are used for data analysis

- 1 RANK & DENSE_RANK
- 2 ROW_NUMBER
- 3 LAG & LEAD
- 4 FIRST_VALUE
- 5 LAST_VALUE
- 6 NTILE

RANK() & DENSE_RANK() :-

=> both functions used to find ranks
=> ranking is based on one or more fields
=> for rank functions data must be sorted

```
RANK() OVER (ORDER BY colname ASC / DESC , -----)
DENSE_RANK() OVER (ORDER BY colname ASC / DESC , -----)
```

Ex :-

=> display ranks of the employees based on sal
highest paid employee should get 1st rank ?

```
SELECT empno,ename,sal,
        RANK() OVER (ORDER BY sal DESC) as rnk
FROM emp
```

```
SELECT empno,ename,sal,
        DENSE_RANK() OVER (ORDER BY sal DESC) as rnk
FROM emp
```

=> difference between rank & dense_rank ?

- 1 in rank function if there is a tie then it will skip next rank
but dense_rank will not skip the next rank
- 2 in rank function ranks may not be in sequence but in dense_rank
ranks are always in sequence.

```
SAL          RANK          DENSE_RANK
```


5000	1	1
4000	2	2
3000	3	3
3000	3	3
3000	3	3
2000	6	4
2000	6	4
1000	8	5

=> find ranks based on experience ?
employee having max experience should get 1st rank ?

```
SELECT empno,ename,hiredate,
       DENSE_RANK() OVER (ORDER BY hiredate ASC) as rnk
FROM emp
```

PARTITION BY clause :-
=====

=> partition by clause is used to divide the table based on one or more fields.

=> used to find ranks with in group , for ex to find ranks with in dept
first divide the table dept wise using partition by clause and
apply rank / dense_rank functions on each partition

Ex :-

```
SELECT empno,ename,sal,deptno,
       DENSE_RANK() OVER (PARTITION BY deptno
                           ORDER BY sal DESC) as
rnk
FROM emp
```

ROW_NUMBER() :-
=====

=> returns record number
=> row_number is also based on one or more fields
=> for row_number also data must be sorted

```
ROW_NUMBER() OVER ([PARTITION BY col1,col2,--]
                   ORDER BY ---)
```

Ex 1 :- row_number based on sal

```
SELECT empno,ename,sal,
       row_number() over (ORDER BY sal DESC) as rno
FROM emp
```

7839	KING	5000.00	1
7902	FORD	3000.00	2
7566	JONES	3000.00	3
7788	SCOTT	3000.00	4
7698	BLAKE	2900.00	5

Ex 2 :- row_number based on empno

```
SELECT empno,ename,sal,
       ROW_NUMBER() OVER (ORDER BY empno ASC) as rno
FROM emp
```

7369	SMITH	800.00	1
------	-------	--------	---

7499	ALLEN	1600.00	2
7521	WARD	1300.00	3
7566	JONES	3000.00	4
7654	MARTIN	1300.00	5

rank vs dense_rank vs row_number :-

=====

SAL	RNK	DRNK	RNO
5000	1	1	1
4000	2	2	2
3000	3	3	3
3000	3	3	4
3000	3	3	5
2000	6	4	6
2000	6	4	7
1000	8	5	8

NOTE :-

=> window functions are not allowed in where clause because window functions are executed after where clause

```
SELECT empno,ename,sal,
       ROW_NUMBER() OVER (ORDER BY empno ASC) as rno
FROM emp
WHERE ROW_NUMBER() OVER (ORDER BY empno ASC) <= 5 => ERROR
```

=> To overcome this use CTE (common table expression)

```
WITH <cte-name>
AS
  (SELECT statement)
```

```
SELECT * FROM <cte-name>
```

Ex :-

=> display first 5 rows from emp table ?

```
WITH E
AS
  (SELECT empno,ename,sal,
         ROW_NUMBER() OVER (ORDER BY empno ASC) as rno
   FROM emp)
```

```
SELECT * FROM E WHERE rno <= 5
```

```
WHERE rno = 5
```

```
WHERE rno IN (5,10,15)
```

```
WHERE rno BETWEEN 5 AND 10
```

```
WHERE rno%2 = 0
```

=> display top 3 max salaries ?

```
WITH E
AS
  (SELECT sal,
         dense_rank() OVER (ORDER BY sal DESC) as rnk
   FROM emp)
SELECT DISTINCT * FROM E WHERE rnk <= 3 ORDER BY sal DESC
```

=> display 5th max salary ?

```
WITH E
AS
(SELECT sal,
  dense_rank() OVER (ORDER BY sal DESC) as rnk
FROM emp)
SELECT DISTINCT * FROM E WHERE rnk = 5
```

=> display top 3 max salaries in each dept ?

```
WITH E
AS
( SELECT sal,deptno,
  dense_rank() over (partition by deptno
                    order by sal desc) as rnk
  FROM emp
)
SELECT deptno,sal,rnk FROM E WHERE rnk <= 3
```

NOTE :-

Queries based on row number then use row_number function
Queries based on top 5 , 5th max salary then use dense_rank function

=>

```
STUDENT
sno  sname m    p    c
1    A    80   90   70
2    B    60   70   50
3    C    90   70   80
4    D    90   80   70
```

1 find ranks of the students based on total marks
if total marks same then ranking should be based on m
if m also same then ranking should be based on p ?

2 find top 3 students based on total marks ?

24-apr-26

LAG () & LEAD() :-
=====

LAG(colname,INT) OVER (ORDER BY ---) => returns previou row value
LEAD(colname,INT) OVER (ORDER BY ---) => returns next row value

Ex 1 :-

```
SELECT empno,ename,sal,
      LAG(sal,1) OVER (ORDER BY empno ASC)
FROM emp
```

```
7369 SMITH 800.00      NULL
7499 ALLEN 1600.00     800.00
7521 WARD  1300.00     1600.00
7566 JONES 3000.00     1300.00
7654 MARTIN      1300.00    3000.00
```

Ex 2 :-

```
CREATE TABLE population
(
  year    INT,
  population NUMERIC
)
```

```
INSERT INTO population VALUES(2020,1328024498) ,
                                (2021,1402617695),
                                (2022,1425423212),
                                (2023,1438069596),
                                (2024,1450935791),
                                (2025,1470935791)
```

```
=> display      YEAR      POPULATION  GROWTH  ?
```

```
SELECT year,population,
       population - LAG(population,1) OVER (ORDER BY year ASC) as
growth
FROM population
```

```
2020  1328024498  NULL
2021  1402617695  74593197
2022  1425423212  22805517
2023  1438069596  12646384
2024  1450935791  12866195
2025  1470935791  20000000
```

```
=> display  YEAR      POPULATION  GROWTH      PCT  ?
```

```
PCT =      ( growth / prev year population ) * 100
```

```
WITH P
AS
(
  SELECT  year, population,
          LAG(population,1) OVER (ORDER BY year ASC) as
prev_year_population
FROM population
)
```

```
SELECT year,population,
       population - prev_year_population as growth,
       (( population - prev_year_population )/
(prev_year_population))*100 as pct
```

```
FROM P
```

```
=> display  ENAME      HIREDATE      DAYS  ?
```

```
SMITH 1980-12-17 null
ALLEN 1981-02-20 60+
WARD  1981-02-22 2
```

```
days =      current hiredate - prev hiredate
```

```
SELECT  ename,hiredate,
        DATEDIFF(DD , LAG(hiredate,1) OVER (ORDER BY hiredate ASC),hiredate)
as days
FROM emp
```

FIRST_VALUE (), LAST_VALUE() :-
=====

FIRST_VALUE(colname) OVER (ORDER BY --) => returns first row value
LAST_VALUE(colname) OVER (ORDER BY --) => returns last row value

Ex :-

```
SELECT ename,sal,  
       FIRST_VALUE(sal) OVER (ORDER BY empno ASC) as first  
FROM emp
```

SMITH	800.00	800.00
ALLEN	1600.00	800.00
WARD	1300.00	800.00
JONES	3000.00	800.00
MARTIN	1300.00	800.00

```
SELECT ename,sal,  
       LAST_VALUE(sal) OVER (ORDER BY empno ASC  
                            ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED
```

```
FOLLOWING) as LAST  
FROM emp
```

SMITH	800.00	1300.00
ALLEN	1600.00	1300.00
WARD	1300.00	1300.00
JONES	3000.00	1300.00
MARTIN	1300.00	1300.00

25-apr-26

Multi-row functions / Aggregate Functions :-
=====

=> these functions process multiple rows and returns one value
=> these functions are also called as aggregate functions

MAX => returns maximum value
MIN => returns minimum value
SUM => returns total
AVG => returns average
COUNT => returns no of values in a column
COUNT(*) => returns no of rows in a table

```
SELECT MAX(sal) FROM emp => 5000  
SELECT MAX(hiredate) FROM emp => 1983-01-12  
SELECT MAX(ename) FROM emp => ward
```

```
SELECT MIN(sal) FROM emp => 800
```

```
SELECT SUM(sal) FROM emp => 29300.00
```

=> round the total sal to thousands ?

```
SELECT ROUND(SUM(sal) , -3) FROM emp => 29000
```

=> after rounding display total sal with thousand separator and currency symbol ?

```
SELECT FORMAT(ROUND(SUM(sal) , -3) , 'c' , 'en-in' ) FROM emp
```

o/p :- â¹ 29,000.00

=> display total sal paid to managers ?

```
SELECT SUM(sal)
FROM emp
WHERE job='manager'   => 8400
```

AVG() :-
=====

=> returns average value

```
AVG(colname)
```

Ex :-

```
SELECT AVG(sal) FROM emp   => 2092.8571
```

=> round avg sal to lowest integer ?

```
SELECT FLOOR(AVG(sal)) FROM emp   => 2092
```

NOTE :-

sum, avg cannot be applied on char, date columns allowed only on numeric columns

difference between analytical and aggregate functions ?

=> analytical functions returns one value per each row

=> aggregate functions returns one value from group of rows

How to use aggregate functions as analytical functions ?

```
SUM(colname) OVER (ORDER BY -----)
AVG(colname) OVER (ORDER BY -----)
```

Ex :-

```
SELECT empno,ename,sal,
       SUM(sal) OVER (ORDER BY empno ASC) as running_total
FROM emp
```

7369	SMITH	800.00	800.00
7499	ALLEN	1600.00	2400.00
7521	WARD	1300.00	3700.00
7566	JONES	3000.00	6700.00
7654	MARTIN	1300.00	8000.00

```
SELECT empno,ename,sal,
       AVG(sal) OVER (ORDER BY empno ASC)
                        as moving_avg
FROM emp
```

7369	SMITH	800.00	800.00
7499	ALLEN	1600.00	1200.00
7521	WARD	1300.00	1233.3333
7566	JONES	3000.00	1675.00

```

7654 MARTIN      1300.00      1600.00
7698 BLAKE 2900.00      1816.6666
7782 CLARK 2500.00      1914.2857

```

```

SELECT empno,ename,sal,
       AVG(sal) OVER (ORDER BY empno ASC
                     ROWS BETWEEN
                     3 PRECEDING AND CURRENT ROW) as
moving_avg
FROM emp

```

```

7369 SMITH 800.00      800.00
7499 ALLEN 1600.00     1200.00
7521 WARD  1300.00     1233.3333
7566 JONES 3000.00     1675.00
7654 MARTIN      1300.00     1800.00
7698 BLAKE 2900.00     2125.00
7782 CLARK 2500.00     2425.00

```

COUNT() :-
=====

=> returns no of values present in a column

COUNT(colname)

ex :-

```

SELECT COUNT(empno) FROM emp      => 14
SELECT COUNT(comm) FROM emp      => 4  => nulls are not counted
SELECT COUNT( deptno) FROM emp   => 14
SELECT COUNT(DISTINCT deptno) FROM emp => 3 => counts only unique values

```

COUNT(*) :-
=====

=> returns no of rows in a table

SELECT COUNT(*) FROM emp => 14

difference between COUNT & COUNT(*) ?

COUNT function ignores nulls
COUNT(*) function includes nulls

```

T1
F1
10
NULL
20
NULL
30

```

```

COUNT(F1)      => 3
COUNT(*)       => 5

```

=> how many employees joined in 1981 year ?

```

SELECT COUNT(*)
FROM emp
WHERE datepart(yy,hiredate) = 1981

```

=> how many employees joined on sunday ?

```
SELECT COUNT(*)
FROM emp
WHERE DATENAME(DW,hiredate) = 'sunday'
```

=> how many employees joined in 2nd quarter of 1981 year ?

```
SELECT COUNT(*)
FROM emp
WHERE DATEPART(yy,hiredate) = 1981 AND DATEPART(QQ,hiredate) = 2
```

NOTE :-

=> aggregate functions are not allowed in where clause and they are allowed only in
select,having clauses

```
SELECT ename
FROM emp
WHERE sal = MAX(sal)    => ERROR
```

```
SELECT deptno
FROM emp
WHERE COUNT(*) > 3    => ERROR
```

summary :-

```
char      :-
upper,lower,len,left,right,substring,charindex,replicate,replace,translate,stuff
date      :-  getdate(),datepart,datetime,dateadd,datediff,eomonth
numeric   :-  abs,power,sqrt,square,round,ceiling,floor
conversion :-  cast,convert
analytical :-  rank, dense_rank,row_number,lag,lead
aggregate :-  max,min,sum,avg,count,count(*)
```

=====

27-apr-26

GROUP BY clause :-
=====

=> GROUP BY clause is used to group rows based on one or more columns to calculate max,min,sum,avg,count for each group. For ex to calculate dept wise total sal first group the rows based on dept and apply sum function on each group i.e. each dept.

emp		empno	ename	sal	deptno			
	1	A	3000	10				
	2	B	4000	20		group by	10	8000
	3	C	5000	30	=====>	20	10000	
	4	D	6000	20				
30	5000							
	5	E	5000	10				

detailed data

summarized data

=> GROUP BY clause converts detailed data to summarized data which is useful for analysis.

syntax :-

```

SELECT columns
FROM tabname
[WHERE cond]
GROUP BY col1,col2, ---
[HAVING cond]
[ORDER BY col1,col2,---]

```

Execution :-

```

FROM
WHERE
GROUP BY
HAVING
SELECT
ORDER BY

```

Ex :-

=> display dept wise total salary ?

```

SELECT deptno,SUM(sal) as totalsal
FROM emp
GROUP BY deptno

```

FROM emp :-

emp			
empno	ename	sal	deptno
1	A	3000	10
2	B	4000	20
3	C	5000	30
4	D	6000	20
5	E	5000	10

GROUP BY deptno :-

10	1	A	3000
	5	E	5000
20	2	B	4000
	4	D	6000
30	3	C	5000

SELECT deptno,SUM(sal) as totalsal :-

10	8000
20	10000
30	5000

=> display dept wise total sal only for the depts 10,20 ?

```

SELECT deptno,SUM(sal) as totalsal
FROM emp
WHERE deptno IN (10,20)
GROUP BY deptno ;

```

FROM emp :-

```
emp
empno ename sal deptno
1      A      3000 10
2      B      4000 20
3      C      5000 30
4      D      6000 20

5      E      5000 10
```

WHERE deptno IN (10,20) :-

```
1      A      3000 10
2      B      4000 20
4      D      6000 20

5      E      5000 10
```

GROUP BY deptno :-

```
10 1      A      3000
   5      E      5000

20 2      B      4000
   4      D      6000
```

SELECT deptno,SUM(sal) :-

```
10      8000
20     10000
```

=> display dept wise total sal
display only the depts sum(sal) greater than 10000 ?

```
SELECT deptno,SUM(sal) as totalsal
FROM emp
WHERE SUM(sal) > 10000
GROUP BY deptno          => ERROR
```

sql server cannot calculate dept wise total sal before group by and it can calculate only after group by , so apply the condition SUM(sal) > 10000 after group by using HAVING clause.

```
SELECT deptno,SUM(sal) as totalsal
FROM emp
GROUP BY deptno
HAVING SUM(sal) >= 10000
```

FROM emp :-

```
emp
empno ename sal deptno
1      A      3000 10
2      B      4000 20
3      C      5000 30
4      D      6000 20

5      E      5000 10
```

GROUP BY deptno :-

10	1	A	3000
	5	E	5000
20	2	B	4000
	4	D	6000
30	3	C	5000

HAVING SUM(sal) >= 10000 :-

20	2	B	4000
	4	D	6000

SELECT deptno,SUM(sal) :-

20	10000
----	-------

where vs having :-

WHERE

HAVING

- | | | |
|---|---|---|
| 1 | to filter rows | to filter groups |
| 2 | executed before group by | executed after group by |
| 3 | where can be used without group by | cannot be used without group by |
| 4 | use where clause if cond doesn't contain aggregate function | use having clause if cond contains aggregate function |

=> display job wise no of employees
only the employees working as clerk,manager
no of employees > 3 ?

```
select job,count(job)
from emp
where job in ('clerk','manager')
group by job
having count(job)>3
```

28-apr-26

=> find the departments having more than 3 employees ?

```
SELECT deptno,COUNT(*) as cnt
FROM emp
GROUP BY deptno
HAVING COUNT(*) > 3
```

20	5
30	6

=> display southern states having more than 5cr population ?

```
PERSONS
aadharo name gender dob addr city state
```

```

SELECT state,count(*)
FROM persons
WHERE state IN ('AP','TG','KL','TN','KA')
GROUP BY state
HAVING COUNT(*) > 50000000

```

=> display year wise no of employees ?

```

SELECT DATEPART(YY,hiredate) as year,COUNT(*) as cnt
FROM emp
GROUP BY DATEPART(YY,hiredate)

```

```

1980  1
1981  10
1982  2
1983  1

```

```

GROUP BY year => invalid
ORDER BY year ASC => valid

```

=> column alias cannot be used in group by because group by is executed before select

=> column alias can be used in order by because order by is executed after select

execution :-

```

FROM
WHERE
GROUP BY
HAVING
SELECT
ORDER BY

```

=> display day wise no of employees joined ?

```

SELECT DATENAME(DW,hiredate) as day,COUNT(*) as cnt
FROM emp
GROUP BY DATENAME(DW,hiredate)
ORDER BY day ASC

```

```

Friday          2
Monday           1
Saturday         1
Sunday           1
Thursday         4
Tuesday          3
Wednesday       2

```

note :-

=> only group by columns and aggregate functions should appear in select other than these columns if any column appears then it returns error

```

SELECT DATENAME(DW,hiredate) as day,ename,COUNT(*) as cnt
FROM emp
GROUP BY DATENAME(DW,hiredate)
ORDER BY day ASC      => ERROR

```

Grouping based on multiple columns :-

=====

=> display dept wise and with in dept job wise total sal ?

```

SELECT deptno,job,SUM(sal) as totalsal
FROM emp
GROUP BY deptno,job
ORDER BY deptno ASC

```

```

10      CLERK      1300
        MANAGER    2450
        PRESIDENT  5000

20      ANALYST    6000
        CLERK      1900
        MANAGER    2975

30      CLERK      950
        MANAGER    2950
        SALESMAN   5600

```

=> display year wise and with in year quarter wise no of employees joined ?

```

1981  1      ?
      2      ?
      3      ?
      4      ?

```

```

SELECT DATEPART(yy,hiredate) as year,
        DATEPART(qq,hiredate) as qrt,
        COUNT(*) as cnt
FROM emp
GROUP BY DATEPART(yy,hiredate) , DATEPART(qq,hiredate)
ORDER BY year ASC , qrt ASC

```

29-apr-26

ROLLUP & CUBE :-
=====

=> both functions used to display subtotals and grand totals

```

GROUP BY ROLLUP( col1,col2,----)
GROUP BY CUBE(col1,col2,-----)

```

Ex :-

ROLLUP :-
=====

=> rollup displays subtotals for each group and also displays grand total

Ex :-

```

SELECT deptno,job,SUM(sal) as totalsal
FROM emp
GROUP BY ROLLUP(deptno,job)
ORDER BY deptno ASC

NULL  NULL      29300.00 => grand total

10    CLERK      1300.00
10    MANAGER    2500.00
10    PRESIDENT  5000.00
10    NULL      8800.00 => dept subtotal

```

20	ANALYST	6000.00	
20	CLERK	1900.00	
20	MANAGER	3000.00	
20	NULL	10900.00	=> dept subtotal
30	CLERK	1000.00	
30	MANAGER	2900.00	
30	SALESMAN	5700.00	
30	NULL	9600.00	=> dept subtotal

CUBE :-

=> cube displays subtotals for each group by column i.e. deptno and job

```
SELECT deptno,job,SUM(sal) as totalsal
FROM emp
GROUP BY CUBE(deptno,job)
ORDER BY deptno ASC
```

NULL	NULL	29300.00	=> grand total
NULL	ANALYST	6000.00	=> job subtotal
NULL	CLERK	4200.00	
NULL	MANAGER	8400.00	
NULL	PRESIDENT	5000.00	
NULL	SALESMAN	5700.00	
10	NULL	8800.00	=> dept subtotal
10	CLERK	1300.00	
10	MANAGER	2500.00	
10	PRESIDENT	5000.00	
20	NULL	10900.00	=> dept subtotal
20	ANALYST	6000.00	
20	CLERK	1900.00	
20	MANAGER	3000.00	
30	NULL	9600.00	=> dept subtotal
30	CLERK	1000.00	
30	MANAGER	2900.00	
30	SALESMAN	5700.00	

difference between rollup & cube ?

=> rollup displays subtotals only for the first column in group by
=> cube displays subtotals for all column in group by

=> display state wise and with in state gender wise total population
and also display state wise and gender wise subtotals ?

```
PERSONS
aadharo name gender dob addr city state
```

summary :-

=> purpose of group by
=> writing queries using group by
=> where vs having
=> rollup vs cube

CASE statement :-
=====

=> CASE statement is used to implement IF-ELSE in sql queries
=> using case statement we can return values based on condition
=> CASE statements are 2 types

- 1 simple case
- 2 searched case

simple case :-
=====

=> use simple case when conditions based on "=" operator

```
CASE colname
WHEN value1 THEN return expr1
WHEN value2 THEN return expr2
-----
[ELSE return expr]
END
```

```
IF col = value1 THEN
    rreturn expr1
ELSE IF col = value2 THEN
    return expr2
-----
ELSE
    return expr
```

=> display ENAME JOB ?

```

                if JOB=CLERK DISPLAY  WORKER
                        MANAGER          BOSS
                        PRESIDENT        BIG BOSS
                        ELSE              EMPLOYEE

SELECT  ename,
        CASE job
        WHEN 'CLERK'   THEN 'WORKER'
        WHEN 'MANAGER' THEN 'BOSS'
        WHEN 'PRESIDENT' THEN 'BIG BOSS'
        ELSE 'EMPLOYEE'
        END as job
FROM emp
```

=> increment salaries as follows ?

```

IF deptno = 10  incr sal by 10%
                20                15%
                30                20%
                others             5%

UPDATE emp
SET sal = CASE deptno
            WHEN 10 THEN sal+(sal*0.1)
            WHEN 20 THEN sal+(sal*0.15)
            WHEN 30 THEN sal+(sal*0.2)
            ELSE  sal + (sal*0.05)
            END
```

searched case :-
=====

=> use searched case when conditions not based on "=" operator

```

CASE
WHEN cond1 THEN return expr1
WHEN cond2 THEN return expr2
-----
ELSE return expr
END

```

=> display ENAME SAL SALRANGE ?

```

IF sal>3000 HISAL
  sal<3000 LOSAL
  else      AVGSAL

```

```

SELECT ename,sal,
       CASE
       WHEN sal>3000 THEN 'HISAL'
       WHEN sal<3000 THEN 'LOSAL'
       ELSE 'AVGSAL'
       END as salrange
FROM emp

```

=> display SNO TOTAL AVG RESULT ?

```

STUDENT
sno sname s1    s2    s3
1  A     80    90    70
2  B     30    60    50

```

=> count no of employees based on salary range ?

```

0-2000  ?
2001-4000  ?
>4000    ?

```

30-apr-26

Integrity Constraints :-
=====

=> Integrity Constraints are rules to maintain Data Integrity i.e. Data Quality or Data Consistency

=> Integrity Constraints are used to prevent users from entering invalid data.

=> Integrity Constraints are used to enforce rules like min bal must be 1000.

Types of constraints :-
=====

- 1 NOT NULL
- 2 UNIQUE
- 3 PRIMARY KEY
- 4 CHECK
- 5 FOREIGN KEY
- 6 DEFAULT

=> above constraints can be declared in two ways

1 column level 2 table level

column level :-
=====

=> if constraints are declared immediately after declaring column then it is called column level

```
CREATE TABLE tabname
(
    colname datatype(size) constraint ,
    colname datatype(size) constraint ,
    -----
)
```

NOT NULL :-
=====

=> NOT NULL constraint doesn't accept null values

Ex :-

```
CREATE TABLE emp11
(
    empno INT ,
    ename VARCHAR(10) NOT NULL
)

INSERT INTO emp11 VALUES(100,NULL)    => ERROR
INSERT INTO emp11 VALUES(101,'A')
```

UNIQUE :-
=====

=> unique constraint doesn't accept duplicates

Ex :-

```
CREATE TABLE cust
(
    custid INT ,
    cname VARCHAR(10) NOT NULL,
    emailid VARCHAR(20) UNIQUE
)

INSERT INTO cust VALUES(100,'A','abc@gmail.com')
INSERT INTO cust VALUES(101,'B','abc@gmail.com') => ERROR
INSERT INTO cust VALUES(102,'C',NULL)
INSERT INTO cust VALUES(103,'D',NULL) => ERROR
```

How to insert multiple nulls :-
=====

```
CREATE TABLE cust
(
    custid INT ,
    cname VARCHAR(10) NOT NULL,
    emailid VARCHAR(20)
)
```

CREATE UNIQUE INDEX I10 ON CUST(emailid) WHERE emailid IS NOT NULL

```
INSERT INTO cust VALUES(100,'A',NULL)
INSERT INTO cust VALUES(101,'B',NULL)
INSERT INTO cust VALUES(102,'C','abc@gmail.com')
INSERT INTO cust VALUES(103,'D','abc@gmail.com') => ERROR
```

PRIMARY KEY :-

=====

=> PRIMARY KEY doesn't accept duplicates and nulls
=> it is the combination of UNIQUE & NOT NULL
=> In tables one column must be there to uniquely identify and into that column
duplicates and nulls are not allowed , so declare that column with primary key.

Ex :-

```
CREATE TABLE emp15
(
  empid INT PRIMARY KEY,
  ename VARCHAR(10) NOT NULL
)
```

```
INSERT INTO emp15 VALUES(100,'A')
INSERT INTO emp15 VALUES(100,'B')      => ERROR
INSERT INTO emp15 VALUES(NULL,'C')    => ERROR
```

note :-

=> only one primary key is allowed per table , if we want multiple primary keys then
declare one column with primary key and other columns with UNIQUE NOT NULL.

Ex :-

```
CREATE TABLE cust
(
  custid INT PRIMARY KEY,
  cname VARCHAR(20) NOT NULL,
  aadharno NUMERIC(12) UNIQUE NOT NULL ,
  panno CHAR(10) UNIQUE NOT NULL
)
```

=> columns declared with UNIQUE NOT NULL are called secondary key columns

CHECK :-

=====

=> use check constraint when rule based on condition

CHECK(condition)

=> if cond = true data is accepted

Ex 1 :- sal must be min 3000

```
CREATE TABLE emp20
(
  empno INT PRIMARY KEY,
  ename VARCHAR(10) NOT NULL,
  sal MONEY CHECK(sal >= 3000)
)
```

```
INSERT INTO emp20 VALUES(100,'A',1000)      => ERROR
INSERT INTO emp20 VALUES(101,'B',5000)
```

```
INSERT INTO emp20 VALUES(102,'C',NULL)
```

=> check constraint allows nulls

Ex 2 :- gender must be 'M','F' ?

```
GENDER CHAR(1) CHECK(GENDER IN ('M','F'))
```

Ex 3 :- amt must be multiple of 100 ?

```
AMT MONEY CHECK(AMT%100=0)
```

Ex 4 :- pwd must be min 6 chars

```
PWD VARCHAR(12) CHECK( LEN(PWD) >= 6)
```

Ex 5 :- emailid must contain '@'
must end with '.com' or '.co' or '.in'

```
emailid VARCHAR(30) CHECK( emailid LIKE '%@%'
AND
(
    emailid
    LIKE '%.com'
    OR
    emailid LIKE
    '%.co'
    OR
    emailid LIKE
    '%.in'
))
```

1-may-26

FOREIGN KEY :-
=====

=> foreign key is used to establish relationship between two tables

=> To establish relationship between two tables then take primary key of one table and add it to another table as foreign key and declare with references constraint.

ex :-

```
projects
projid  pname duration  cost  client
100 AAA   5      300   DBS Bank
101 BBB   3      200   Airlines
```

```
emp
empid   ename job   sal  projid REFERENCES projects(projid)
1      A   SE    30K    100
2      B   SSE   60K    101
3      C   SE    40K    999 => ERROR
4      D   SSE   80K    100
5      E   SE    25K    NULL
```

=> values entered in fk column should match with values entered in pk column

=> fk allows duplicates and nulls

=> after declaring foreign key a relationship is established between two tables called parent / child relationship .

=> pk table is parent and fk table is child

```
CREATE TABLE projects
(
    projid INT PRIMARY KEY,
    pname VARCHAR(10) NOT NULL
)

INSERT INTO projects VALUES(100, 'DBS'), (101, 'INDAL')
```

```
CREATE TABLE emp_proj
(
    empid INT PRIMARY KEY,
    ename VARCHAR(10) NOT NULL ,
    sal MONEY CHECK(sal>=5000),
    projid INT REFERENCES projects(projid)
)
```

```
INSERT INTO emp_proj VALUES(1, 'A', 10000, 100)
INSERT INTO emp_proj VALUES(2, 'B', 20000, 999)    => ERROR
INSERT INTO emp_proj VALUES(3, 'C', 30000, 100)
INSERT INTO emp_proj VALUES(4, 'D', 10000, NULL)
```

Relationship Types :-
=====

- 1 one to one (1:1)
- 2 one to many (1:m)
- 3 many to one (m:1)
- 4 many to many (m:n)

=> by default sql server creates one to many relationship between two tables

How to establish one to one relationship :-

=> To establish one to one relationship then declare fk with unique constraint

ex :-

```
DEPT
dno dname
10 HR
20 IT
30 SALES
```

```
EMP
empno  ename sal      dno REFERENCES dept(dno)
1  A    40000    10
2  B    30000    10
3  C    20000    10
```

```
MGR
mgrno  mname dno REFERENCES dept(dno) UNIQUE
2      B    10
```

How to establish many to many relationship :-
=====

=> To establish many to many relationship then create 3rd table
and add primary keys of both tables as foreign keys.

Ex :-

```
STUDENT          COURSE
sid  sname       cid  cname
```

1	A	10	.NET
2	B	11	SQL SERVER

REGISTRATIONS

sid	cid	dor	fee
1	10	?	?
1	11	?	?
2	10	?	?
2	11	?	?

DEFAULT :-

=====

=> a column can be declared with default value as follows

Ex :- HIREDATE DATE DEFAULT GETDATE

=> while inserting if we skip hiredate then sql server inserts default value

```
CREATE TABLE emp22
(
    empno INT PRIMARY KEY,
    ename VARCHAR(10) NOT NULL,
    hiredate DATE DEFAULT GETDATE()
)

INSERT INTO emp22(empno,ename) VALUES(100,'A')
INSERT INTO emp22 VALUES(101,'B','2026-01-01')
INSERT INTO emp22 VALUES(102,'C',NULL)
```

SELECT * FROM emp22

100	A	2026-05-01
101	B	2026-01-01
102	C	NULL

=> create following tables with given rules ?

ACCOUNTS

accno	actype	bal
-------	--------	-----

Rules :-

- 1 accno should not be duplicate & null
- 2 actype must be 'S' OR 'C'
- 3 bal must be min 1000

TRANSACTIONS

trid	ttype	tdate	tamt	accno
------	-------	-------	------	-------

Rules :-

- 1 trid should not be duplicate & null
- 2 ttype must be 'W' OR 'D'
- 3 tdate must be system date
- 4 tamt must be multiple of 100
- 5 accno should match with accounts table accno

2-may-26

TABLE LEVEL :-

=====

=> if constraints are declare after declaring all columns then it is called table level

=> use table level to declare constraints for combination of columns

Declaring check constraint at table level :-

=====

```
products
prodid  pname price mfd_dt      exp_dt
100      AAA      50  2026-05-01  2026-04-01 => invalid
```

Rule :- exp_dt > mfd_dt

```
CREATE TABLE products
(
  prodid INT PRIMARY KEY,
  pname VARCHAR(10) NOT NULL,
  price SMALLMONEY,
  mfd_dt DATE ,
  exp_dt DATE ,
          CHECK(exp_dt > mfd_dt)
)
```

```
INSERT INTO products VALUES(100,'A',50,getdate(),'2026-04-01') => error
INSERT INTO products VALUES(101,'B',50, '2026-04-01',getdate())
```

Composite primary key :-

=> if combination of columns declared primary key then it is called composite primary key

=> In some tables we may not be able to uniquely identify the records using single column and we need combination of columns to uniquely identify and that combination should be declare primary key at table level.

Ex :-

STUDENT		COURSE	
sid	sname	cid	cname
1	A	10	.NET
2	B	11	SQL SERVER

REGISTRATIONS			
sid	cid	dor	fee
1	10	?	?
1	11	?	?
2	10	?	?
2	11	?	?

```
CREATE TABLE student
(
  sid INT PRIMARY KEY,
  sname VARCHAR(10) NOT NULL
)
```

```
INSERT INTO student VALUES(1,'A') , (2,'B')
```

```
CREATE TABLE course
(
  cid INT PRIMARY KEY,
  cname VARCHAR(10) NOT NULL
```

)

```
INSERT INTO course VALUES(10, '.NET'), (11, 'SQL')
```

```
CREATE TABLE registrations
```

```
(
  sid INT REFERENCES student(sid) ,
  cid INT REFERENCES course(cid) ,
  dor DATE,
  fee SMALLMONEY ,
  PRIMARY KEY(sid,cid)
)
```

```
INSERT INTO registrations VALUES(1,10,getdate(),5000)
```

```
INSERT INTO registrations VALUES(1,11,getdate(),3000)
```

```
INSERT INTO registrations VALUES(2,10,getdate(),5000)
```

```
INSERT INTO registrations VALUES(1,10,getdate(),5000) => error
```

which of the following constraint cannot be declared at table level ?

- A UNIQUE
- B CHECK
- C NOT NULL
- D PRIMARY KEY
- E FOREIGN KEY

ANS :- C

Which statements are true regarding constraints ?

- A a foreign key cannot contain NULL value F
- B a column with UNIQUE constraint can contain NULL value T
- C a constraint is enforced only for the INSERT operation on a table F
- D all constraints can be defined at column level and table level. F

Which CREATE TABLE statement is valid?

A. CREATE TABLE ord_details

```
(ord_no INT PRIMARY KEY,
item_no INT PRIMARY KEY,
ord_date DATE NOT NULL);
```

B. CREATE TABLE ord_details

```
(ord_no INT UNIQUE , NOT NULL,
item_no INT,
ord_date DATE DEFAULT GETDATE() NOT NULL);
```

C. CREATE TABLE ord_details

```
(ord_no INT ,
item_no INT,
ord_date DATE DEFAULT NOT NULL,
UNIQUE (ord_no),
PRIMARY KEY (ord_no));
```

D. CREATE TABLE ord_details

```
(ord_no INT,
item_no INT,
ord_date DATE DEFAULT GETDATE() NOT NULL,
PRIMARY KEY (ord_no, item_no));
```

DELETE rules :-

=====

1 on delete no action (default)

2 on delete cascade
3 on delete set null
4 on delete set default

=> these rules are declared with foreign key.

=> delete rule specifies how child rows are affected if parent row is deleted

ON DELETE NO ACTION :-

=> parent row cannot be deleted if associated with child rows

```
CREATE TABLE dept55
(
  dno INT PRIMARY KEY,
  dname VARCHAR(10)
)
```

```
INSERT INTO dept55 VALUES(10, 'HR') , (20, 'IT')
```

```
CREATE TABLE emp55
(
  empno INT PRIMARY KEY,
  ename VARCHAR(10) NOT NULL,
  dno INT REFERENCES dept55(dno)
)
```

```
INSERT INTO emp55 VALUES(1, 'A', 10) , (2, 'B', 10)
```

```
SELECT * FROM dept55
```

```
10  hr
20  it
```

```
SELECT * FROM emp55
```

```
1  A    10
2  B    10
```

```
DELETE FROM dept55 WHERE dno = 10    => ERROR
```

4-may-26

ON DELETE CASCADE :-

=====

=> if parent row is deleted then it is deleted along with child rows

```
CREATE TABLE dept55
(
  dno INT PRIMARY KEY,
  dname VARCHAR(10)
)
```

```
INSERT INTO dept55 VALUES(10, 'HR') , (20, 'IT')
```

```
CREATE TABLE emp55
(
  empno INT PRIMARY KEY,
  ename VARCHAR(10) NOT NULL,
  dno INT REFERENCES dept55(dno)
          ON DELETE CASCADE
)
```



```
INSERT INTO emp55 VALUES(1, 'A',10) , (2, 'B',10)
```

```
DELETE FROM dept55 WHERE dno = 10 => 1 row affected
```

```
SELECT * FROM emp55 => no rows
```

scenario :-

ACCOUNTS

accno	actype	bal
100		
101		

LOANS

id	type	amt	accno	REFERENCES accounts(accno)
1	H	30	100	
2	C	10	100	

TRANSACTIONS

trid	ttype	tdate	tamt	accno	REFERENCES accounts(accno)
1	W		2000	101	ON DELETE CASCADE
2	D		5000	101	

rules :-

- 1 account cannot be closed if associated with loans
- 2 if account is closed then along with account delete transactions also

ON DELETE SET NULL :-

=====

=> if parent row is deleted then it is deleted without deleting child rows
but fk set to null.

```
CREATE TABLE dept55
```

```
(  
  dno INT PRIMARY KEY,  
  dname VARCHAR(10)  
)
```

```
INSERT INTO dept55 VALUES(10, 'HR') , (20, 'IT')
```

```
CREATE TABLE emp55
```

```
(  
  empno INT PRIMARY KEY,  
  ename VARCHAR(10) NOT NULL,  
  dno INT REFERENCES dept55(dno)  
      ON DELETE SET NULL  
)
```

```
INSERT INTO emp55 VALUES(1, 'A',10) , (2, 'B',10)
```

```
DELETE FROM dept55 WHERE dno = 10 => 1 row affected
```

```
SELECT * FROM emp55
```

empno	ename	dno
1	A	NULL
2	B	NULL

scenario :-

```

projects
projid  pname duration
1000
1001

```

```

emp
empid    ename sal  projid REFERENCES projects(projid)
100      1000 1000  ON DELETE SET NULL
101      1001 1001

```

rule :- if project is completed set the employee projid to null

```

ON DELETE SET DEFAULT :-
=====

```

=> if parent row is deleted then it is deleted without deleting child rows
but fk set to default value

```

CREATE TABLE dept55
(
  dno INT PRIMARY KEY,
  dname VARCHAR(10)
)

```

```

INSERT INTO dept55 VALUES(10, 'HR') , (20, 'IT')

```

```

CREATE TABLE emp55
(
  empno INT PRIMARY KEY,
  ename VARCHAR(10) NOT NULL,
  dno INT DEFAULT 20
          REFERENCES dept55(dno)
          ON DELETE SET DEFAULT
)

```

```

INSERT INTO emp55 VALUES(1, 'A', 10) , (2, 'B', 10)

```

```

DELETE FROM dept55 WHERE dno = 10 => 1 row affected

```

```

SELECT * FROM emp55

```

```

empno    ename dno
1  A      20
2  B      20

```

```

=====
JOINS :-
=====

```

=> join is an operation performed to display data from two or more tables

=> In databases related data stored in multiple tables , to gather or to
combine
data stored in multiple tables we need to join those tables.

ex :-

```

orders
ordid    orddt deltd cid    cust
cid      name  addr  phone
1000      10    10    a      hyd
1001      11    11    b      hyd
1002      12    12    c      hyd

```

output :-

ordid	orddt	deldt	cid	name	addr	phone
1000			10	a	hyd	?

Types of joins :-

=====

- 1 inner join
 - equi join
 - non equi join
 - self join
- 2 outer join
 - left join
 - right join
 - full join
- 3 cross join / cartesian join

EQUI JOIN :-

=====

=> To perform equi join between the two tables there must be a common field and name of the common field need not to be same and pk-fk relationship is not compulsory.

```
SELECT columns
FROM tab1 INNER JOIN tab2
ON join condition
```

join condition :-

=====

=> join condition specifies which record of table1 joined with which record of table2

=> according to given join condition sql server joins the records of two tables

table1.commonfield = table2.commonfield

Ex :-

EMP				DEPT		
empno	ename	sal	deptno	deptno	dname	loc
1	A	5000	10	10	ACCOUNTS	NEW YORK
2	B	3000	20	20	RESEARCH	
3	C	4000	30	30	SALES	
4	D	3000	20	40	OPERATIONS	
5	E	2000	NULL			

=> display employee details with dept details ?

```
SELECT empno,ename,sal,dname,loc
FROM emp INNER JOIN dept
ON emp.deptno = dept.deptno
```

1	A	5000	ACCOUNTS	NEW YORK
2	B	3000	RESEARCH	?
3	C	4000	SALES	?
4	D	3000	RESEARCH	?

5-may-26

=> display employee details with dept details ?

display deptno ?

```
SELECT empno,ename,sal,deptno,dname,loc
FROM emp INNER JOIN dept
ON emp.deptno = dept.deptno    => ERROR
```

=> In join queries declare table alias and prefix column names with table alias for 2 reasons

- 1 to avoid ambiguity
- 2 for faster execution

```
SELECT e.empno,e.ename,e.sal,
       d.deptno,d.dname,d.loc
FROM emp as e INNER JOIN dept as d
ON e.deptno = d.deptno
```

=> display employee details with dept details working at NEW YORK loc ?

```
SELECT e.ename,d.dname,d.loc
FROM emp as e INNER JOIN dept as d
ON e.deptno = d.deptno /* join cond */
WHERE d.loc = 'NEW YORK' /* filter cond */
```

=> display order details with cust details ?
display only the orders to be delivered today ?

orders				cust			
ordid	orddt	deltd	cid	cid	name	addr	phone
1000		10	10	a	hyd		
1001		11	11	b	hyd		
1002		12	12	c	hyd		

```
SELECT o.* , c.*
FROM orders as o INNER JOIN cust as c
ON o.cid = c.cid
WHERE o.deltd = FORMAT(getdate() , 'yyyy-MM-dd')
```

joining more than 2 tables :-

=> if no of tables increases no of join conditions also increases
=> to join N tables N-1 join conditions required

```
SELECT columns
FROM tab1
INNER JOIN tab2
ON join cond
INNER JOIN tab3
ON join cond
INNER JOIN tab4
ON join cond
[where cond]
```

((tab1 join tab2) join tab3) join tab4

Ex :-

EMPLOYEES	DEPARTMENTS	LOCATIONS	COUNTRIES
employee_id		department_id	location_id
country_id			
first_name		department_name	city
country_name			
last_name		location_id	state

salary
department_id

country_id

=> display ENAME DNAME CITY STATE COUNTRY ?

EMPLOYEES DEPARTMENTS LOCATIONS COUNTRIES

```
SELECT e.first_name as ename,
       d.department_name as dname,
       l.city, l.state,
       c.country_name as country
FROM employees as e
INNER JOIN departments as d
ON e.department_id = d.department_id
INNER JOIN locations as l
ON d.location_id = l.location_id
INNER JOIN countries as c
ON l.country_id = c.country_id ;
```

```
SELECT e.first_name as ename,
       d.department_name as dname,
       l.city, l.state,
       c.country_name as country
FROM employees e
INNER JOIN departments d
ON e.department_id = d.department_id
INNER JOIN locations l
ON d.location_id = l.location_id
INNER JOIN countries c
ON l.country_id = c.country_id ;
```

OUTER JOIN :-
=====

=> equi join returns only matching rows but it will not return unmatched rows
=> to display unmatched rows perform outer join

EMP				DEPT			
empno	ename	sal	deptno	deptno	dname	loc	
1	A	5000	10	ACCOUNTS	NEW YORK		
2	B	3000	20	RESEARCH			
3	C	4000	30	SALES			
4	D	3000	20	40	OPERATIONS	=> unmatched row	
5	E	2000	NULL			=> unmatched row	

=> outer join is 3 types

1 left join
2 right join
3 full join

left join :-
=====

=> returns all rows from left side table and matching rows from right side table

```
SELECT e.ename, d.dname
FROM emp as e
LEFT JOIN dept as d
ON e.deptno = d.deptno
```

=> returns all rows from emp and matching rows from dept

A	ACCOUNTS	
B	RESEARCH	
C	SALES	
D	RESEARCH	
E	NULL	=> unmatched from emp

RIGHT JOIN :-

=====

=> returns all rows (matched + unmatched) from right side table and matching rows from left side table

```
SELECT e.ename,d.dname
FROM emp as e
RIGHT JOIN dept as d
ON e.deptno = d.deptno
```

=> returns all rows from dept table and matching rows from emp

A	ACCOUNTS	
B	RESEARCH	
C	SALES	
D	RESEARCH	
NULL	OPERATIONS	=> unmatched from dept

FULL JOIN :-

=====

=> returns all rows from both tables

```
SELECT e.ename,d.dname
FROM emp as e
FULL JOIN dept as d
ON e.deptno = d.deptno
```

A	ACCOUNTS	
B	RESEARCH	
C	SALES	
D	RESEARCH	
E	NULL	=> unmatched from emp
NULL	OPERATIONS	=> unmatched from dept

Displaying only unmatched rows :-

=====

left side table :-

=====

```
SELECT e.ename,d.dname
FROM emp as e
LEFT JOIN dept as d
ON e.deptno = d.deptno
WHERE d.dname IS NULL
```

E	NULL	
---	------	--

right side table :-

=====

```
SELECT e.ename,d.dname
FROM emp as e
RIGHT JOIN dept as d
```

```
ON e.deptno = d.deptno
WHERE e.ename IS NULL
```

NULL OPERATIONS

both tables :-

=====

```
SELECT e.ename,d.dname
FROM emp as e
FULL JOIN dept as d
ON e.deptno = d.deptno
WHERE d.dname IS NULL
      OR
      e.ename IS NULL
```

6-may-26

=> no of rows return by following operations ?

```
equi join =>      8
left join  =>     10
right join =>     10
full join  =>     12
```

T1	T2
F1	F1
1	1
2	2
1	1
2	2
NULL	NULL
NULL	NULL

Non Equi join :-

=====

=> Non equi join is performed between the tables not sharing a common field.

=> here join condition is not based on "=" operator and it is based on
> < between operators.

Ex :-

EMP			SALGRADE		
empno	ename	sal	grade	losal	hisal
1	A	2000	1	700	1000
2	B	5000	2	1001	2000
3	C	1000	3	2001	3000
4	D	3000	4	3001	4000
5	E	1500	5	4001	9999

=> display ENAME SAL GRADE ?

```
SELECT e.ename,e.sal,s.grade
FROM emp as e INNER JOIN salgrade as s
ON e.sal BETWEEN s.losal AND s.hisal
```

A	2000	2
B	5000	5
C	1000	1
D	3000	3
E	1500	2

=> display grade 3 employee list ?

```
SELECT e.ename,e.sal,s.grade
FROM emp as e INNER JOIN salgrade as s
ON e.sal BETWEEN s.losal AND s.hisal
WHERE s.grade = 3
```

=> display ENAME DNAME GRADE ?

```
SELECT e.ename,d.dname,s.grade
FROM emp as e
INNER JOIN dept as d
ON e.deptno = d.deptno
INNER JOIN salgrade as s
ON e.sal BETWEEN s.losal AND s.hisal
```

(emp join salgrade) join dept

on e.sal between s.losal AND s.hisal :-

EMP				SALGRADE			
empno	ename	sal	deptno	grade	losal	hisal	
1	A	2000	10	1	700	1000	
2	B	5000	20	2	1001	2000	
3	C	1000	30	3	2001	3000	
4	D	3000	10	4	3001	4000	
5	E	1500	20	5	4001	9999	

result1				dept		
empno	ename	sal	deptno	grade	deptno	dname
1	A	2000	10	2	10	ACCOUNTS
2	B	5000	20	5	20	RESEARCH
3	C	1000	30	1	30	SALES
4	D	3000	10	3	40	OPERATIONS
5	E	1500	20	2		

on e.deptno = d.deptno :-

1	A	2000	10	2	10	ACCOUNTS
2	B	5000	20	5	20	RESEARCH
3	C	1000	30	1	30	SALES
4	D	3000	10	3	10	ACCOUNTS
5	E	1500	20	2	20	RESEARCH

SELECT :-

A	ACCOUNTS	2
B	RESEARCH	5
C	SALES	1
D	ACCOUNTS	3
E	RESEARCH	2

SELF JOIN :-

=====

=> joining a table to itself is called self join

=> in self join a record in one table joined with another record of same table

=> To perform self join the same table declared two times with different alias

in

FROM clause

Ex :- FROM emp as x INNER JOIN emp as y

emp x			emp y		
empno	ename	mgr	empno	ename	mgr
7369	smith	7902	7369	smith	7902
7499	allen	7698	7499	allen	7698
7566	jones	7839	7566	jones	7839
7698	blake	7839	7698	blake	7839
7839	king	null	7839	king	null
7902	ford	7566	7902	ford	7566

=> display ENAME MGRNAME ?

```
SELECT x.ename,y.ename
FROM emp as x INNER JOIN emp as y
ON x.mgr = y.empno
```

```
SMITH FORD
ALLEN BLAKE
JONES KING
BLAKE     KING
FORD JONES
```

=> display employees reporting to blake ?

```
SELECT x.ename,y.ename as manager
FROM emp as x INNER JOIN emp as y
ON x.mgr = y.empno
WHERE y.ename='BLAKE'
```

=> display blake's manager name ?

```
SELECT x.ename,y.ename as manager
FROM emp as x INNER JOIN emp as y
ON x.mgr = y.empno
WHERE x.ename='BLAKE'
```

=> employees earning more than their managers ?

```
SELECT x.ename,y.ename as manager
FROM emp as x INNER JOIN emp as y
ON x.mgr = y.empno
WHERE x.sal > y.sal
```

=> employees who are joined before their manager ?

```
SELECT x.ename,y.ename as manager
FROM emp as x INNER JOIN emp as y
ON x.mgr = y.empno
WHERE x.hiredate < y.hiredate
```

7-may-26

cross join / cartesian join :-

=====

=> cross join returns cross product or cartesian product of two tables

```
A = 1,2
B = 3,4
```

AXB = (1,3) (1,4) (2,3) (2,4)

=> In cross join all records of 1st table joined with all records of 2nd table
=> for cross join join condition is not required

Ex 1 :-

```
SELECT e.ename,d.dname
FROM emp as e CROSS JOIN dept as d
```

Ex 2 :-

T1			T2		
id	name	price	id	name	price
1	burger	80	1	thumsup	40
2	pizza	120	2	juice	60

output :-

```
burger + thumsup    120
burger + juice    140
pizza + thumsup    160
pizza + juice    180
```

```
SELECT T1.name + ' ' + T2.name as item ,
       T1.price + T2.price as total
FROM T1 CROSS JOIN T2
```

=====

SET OPERATORS :-

=====

```
UNION
UNION ALL
INTERSECT
EXCEPT
```

A = 1,2,3,4
B = 1,2,5,6

```
A UNION B => 1,2,3,4,5,6
A UNION ALL B => 1,2,3,4,1,2,5,6
A INTERSECT B => 1,2
A EXCEPT B => 3,4
B EXCEPT A => 5,6
```

=> In SQL , set operations performed between the rows return by two queries

```
SELECT statement 1
UNION / UNION ALL / INTERSECT / EXCEPT
SELECT statement 2
```

rules :-

- 1 no of columns return by both queries must be same
- 2 corresponding columns datatype must be same

Query 1 :-

```
SELECT job FROM emp WHERE deptno = 20
```

CLERK
MANAGER
ANALYST
CLERK
ANALYST

Query 2 :-

SELECT job FROM emp WHERE deptno = 30

SALESMAN
SALESMAN
SALESMAN
MANAGER
SALESMAN
CLERK

UNION :-

=> combines rows return by two queries
=> duplicates are eliminated
=> result is sorted

SELECT job FROM emp WHERE deptno = 20

UNION

SELECT job FROM emp WHERE deptno = 30

ANALYST
CLERK
MANAGER
SALESMAN

=> difference between UNION & JOIN ?

UNION

JOIN

- | | |
|--|--|
| 1 combines rows | combines columns |
| 2 horizontal merge | vertical merge |
| 3 performed between two similar structures | perofmed between two dissimilar structures |

ex :-

T1	T2
F1	F1
1	10
2	20
3	30

UNION :-

1	1	10
2	2	20
3	3	30
10		
20		
30		

JOIN :-

Ex 2 :-

EMP_IND			
empno	ename	sal	dno
100	A	3000	10
101			20

DEPT				
empno	ename	sal	dno	dname
			10	HR
200		10	20	IT
201		20		

=> total employees list ?

```
SELECT * FROM emp_ind
UNION
SELECT * FROM emp_us
```

=> employees working at US loc with dept details ?

```
SELECT e.* , d.*
FROM emp_us as e INNER JOIN dept as d
ON e.dno = d.dno
```

=> total employees with dept details ?

```
SELECT e.* , d.*
FROM emp_us as e INNER JOIN dept as d
ON e.dno = d.dno
UNION
SELECT e.* , d.*
FROM emp_ind as e INNER JOIN dept as d
ON e.dno = d.dno
```

OR

```
WITH E
AS (
SELECT * FROM emp_ind
UNION
SELECT * FROM emp_us)

SELECT E.*,D.*
FROM E INNER JOIN DEPT AS D
ON E.DNO = D.DNO
```

UNION ALL :-
=====

=> combines rows return by two queries
=> duplicates are not eliminated
=> result is not sorted

Ex :-

```
SELECT job FROM emp WHERE deptno = 20
UNION ALL
SELECT job FROM emp WHERE deptno = 30

CLERK
MANAGER
ANALYST
CLERK
ANALYST
```

SALESMAN
SALESMAN
SALESMAN
MANAGER
SALESMAN
CLERK

=> difference between UNION & UNION ALL ?

	union	union all
--	-------	-----------

- | | | |
|---|---------------------------|----------------------|
| 1 | duplicates are eliminated | not eliminated |
| 2 | result is sorted | result is not sorted |
| 3 | slower | faster |

INTERSECT :-
=====

=> returns common values from result of two queries

```
SELECT job FROM emp WHERE deptno = 20
INTERSECT
SELECT job FROM emp WHERE deptno = 30
```

CLERK
MANAGER

EXCEPT :-
=====

=> returns values present in 1st query output and not present in 2nd query output

```
SELECT job FROM emp WHERE deptno = 20
EXCEPT
SELECT job FROM emp WHERE deptno = 30
```

ANALYST

```
SELECT job FROM emp WHERE deptno = 30
EXCEPT
SELECT job FROM emp WHERE deptno = 20
```

SALESMAN

ex :-

T1	T2
F1	F1
1	1
2	2
3	3
10	40
20	50
30	60

=> write output for following operations ?

- 1 equi join
- 2 left join
- 3 right join
- 4 full join

```
5 union
6 union all
7 intersect
8 except
```

=====

9-may-26

SUB-QUERIES / NESTED QUERIES :-
=====

```
=> a query in another query is called sub-query or nested query
=> one query is called inner / child / sub-query
=> other query is called outer / parent / main
=> first sql server executes inner query next it executes outer query
=> inner query output is input to outer query
=> we can write sub-queries in
```

```
1 WHERE clause
2 HAVING clause
3 FROM clause & CTEs
4 SELECT clause
```

sub-queries in WHERE clause :-
=====

```
SELECT columns
FROM tabname
WHERE colname OP (SELECT stmt) ;
```

```
=> sub-query output acts like a value for outer query
=> use sub-query when where condition based on unknown value.
```

Types of sub-queries :-

```
1 single row sub-queries
2 multi row sub-queries
3 co-related sub-queries
```

single row sub-queries :-
=====

```
=> if inner query returns one value then it is called single row sub-query
=> OP must be any relational operator like > >= < <= = <>
```

Ex :-

```
=> display employees earning more than blake ?
```

```
SELECT *
FROM emp
WHERE sal > ( SELECT sal FROM emp WHERE ename='BLAKE' ) ;
```

2850

```
=> employees who are senior to king ?
```

```
SELECT *
FROM emp
WHERE hiredate < ( SELECT hiredate FROM emp WHERE ename='KING' )
```

1981-11-17

=> display name of the employee earning max salary ?

```
SELECT ename
FROM emp
WHERE sal = MAX(sal) => ERROR
```

aggregate functions not allowed in where clause and they are allowed only in
SELECT, HAVING
clauses

```
SELECT ename
FROM emp
WHERE sal = (SELECT MAX(sal) FROM emp)
```

=> display 2nd max salary ?

```
SELECT MAX(sal)
FROM emp
WHERE sal <> (SELECT MAX(sal) FROM emp) => 3000
```

=> name of the employee earning 2nd max salary ?

```
SELECT ename, sal
FROM emp
WHERE sal = (SELECT MAX(sal)
              FROM emp
              WHERE sal <> (SELECT MAX(sal) FROM emp))
```

=> display name of the employee having max experience ?

```
SELECT ename
FROM emp
WHERE hiredate = (SELECT MAX(hiredate) FROM emp)
```

=> employee names working at NEW YORK loc ?

SUB-QUERY :-

```
SELECT ename
FROM emp
WHERE deptno = (SELECT deptno FROM dept WHERE loc='NEW YORK')
```

JOIN :-

```
SELECT e.ename
FROM emp as e INNER JOIN dept as d
ON e.deptno = d.deptno
WHERE d.loc = 'NEW YORK'
```

=> display ENAME DNAME working at NEW YORK loc ?

JOIN :-

```
SELECT e.ename, d.dname
FROM emp as e INNER JOIN dept as d
```

```
ON e.deptno = d.deptno
WHERE d.loc = 'NEW YORK'
```

SUB-QUERY :-

not possible

difference between sub-query and join ?

1 to display data from one table and condition based on different table then we can use

sub-query or join

2 to display data from two tables then use join

Multi-row sub-query :-

=====

=> if inner query returns multiple values then sub-query is called multi-row sub-query

```
SELECT columns
FROM tablename
WHERE colname OP (SELECT statement)
```

=> OP must be IN NOT IN ANY ALL

single	multi
=	IN
<>	NOT IN
>	>ANY >ALL
<	<ANY <ALL

=> employees working at NEW YORK,CHICAGO locations ?

```
SELECT ename
FROM emp
WHERE deptno IN (SELECT deptno FROM dept WHERE loc IN ('NEW
YORK', 'CHICAGO'))
```

ANY & ALL operators :-

=====

=> these two operators are used for > < comparison with multiple values

```
WHERE sal > ANY(1000,2000,3000)          WHERE sal < ANY (1000,2000,3000)
```

if sal = 800	FALSE	if sal = 800	TRUE
1500	TRUE	1500	TRUE
4000	TRUE		

4000 FALSE

```
WHERE sal > ALL(1000,2000,3000)
```

```
WHERE sal < ALL(1000,2000,3000)
```


	if sal = 800	FALSE		if sal = 800
TRUE				
	1500	FALSE		
1500	FALSE			
	4000	TRUE		
4000	FALSE			

=> employees earning more than all managers ?

```
SELECT *
FROM emp
WHERE sal > ALL ( SELECT sal
                  FROM emp
                  WHERE job='MANAGER')
```

=> employees earning more than atleast one manager ?

```
SELECT *
FROM emp
WHERE sal > ANY ( SELECT sal
                  FROM emp
                  WHERE job='MANAGER')
```

11-may-26

co-related sub-queries :-
=====

=> if inner query references values of outer query then it is called co-related sub-query.

=> execution starts from outer query and inner query is executed no of times depends on
no of rows return by outer query

=> use co-related sub-queries to excute sub-query for each row return by outer query

Ex :-

EMP			
empno	ename	sal	dno
1	A	5000	10
2	B	3000	20
3	C	4000	30
4	D	6000	20
5	E	3000	10

=> employees earning more than avg sal ?

```
SELECT *
FROM emp
WHERE sal > ( SELECT AVG(sal) FROM emp )
```

4200

EMP				
empno	ename	sal	dno	
1	A	5000	10	5000 > (4200) true
2	B	3000	20	3000 > (4200) false
3	C	4000	30	4000 > (4200) false
4	D	6000	20	6000 > (4200) true
5	E	3000	10	3000 > (4200) false

=> employees earning more than avg sal of their dept ?

```
SELECT *
FROM emp as x
WHERE sal > (SELECT AVG(sal) FROM emp WHERE deptno = x.deptno)
```

EMP					
empno	ename	sal	deptno		
1	A	5000	10	5000 > (4000)	TRUE
2	B	3000	20	3000 > (4500)	FALSE
3	C	4000	30	4000 > (4000)	FALSE
4	D	6000	20	6000 > (4500)	TRUE
5	E	3000	10	3000 > (4000)	FALSE

=> display top 3 max salaries ?

method 1 :-

```
SELECT DISTINCT TOP 3 sal
FROM emp
ORDER BY sal DESC
```

method 2 :-

```
SELECT DISTINCT a.sal
FROM emp as a
WHERE 3 > ( SELECT COUNT(DISTINCT b.sal)
            FROM emp as b
            WHERE a.sal < b.sal)
ORDER BY a.sal DESC
```

emp a	emp b		
sal	sal		
5000	5000	3 > (0)	TRUE
3000	3000	3 > (1)	TRUE
2975	2975	3 > (2)	TRUE
1000	1000	3 > (4)	FALSE
2000	2000	3 > (3)	FALSE

=> display 3rd max salary ?

```
SELECT DISTINCT a.sal
FROM emp as a
WHERE (3-1) = ( SELECT COUNT(DISTINCT b.sal)
                FROM emp as b
                WHERE a.sal < b.sal)
ORDER BY a.sal DESC
```

12-may-26

Derived Tables :-

=====

=> sub-queries in FROM clause are called derived tables

```
SELECT columns
FROM (SELECT statement) as <alias>
[WHERE cond]
```

=> sub-query output acts like a table for outer query

=> use derived table for the following scenarios

- 1 to control order of execution of clauses
- 2 to use result of one operation in another operation
- 3 to join two query outputs

controlling order of execution of clauses :-
=====

default order :-
=====

FROM
WHERE
GROUP BY
HAVING
SELECT
ORDER BY

=> To control this order of execution use derived table

```
SELECT
SELECT
FROM      =====>      FROM (SELECT
WHERE      FROM
ORDER BY      ORDER BY )
WHERE
```

Ex 1 :-

=> display ranks of the employees based on sal and highest paid should get 1st rank ?

```
SELECT empno,ename,sal,
          dense_rank() over (order by sal desc) as rnk
FROM emp
```

above query displays ranks of all the employees but to display top 5 employees

```
SELECT empno,ename,sal,
          DENSE_RANK() OVER (ORDER BY sal DESC) as rnk
FROM emp
WHERE rnk <= 5    => ERROR
```

column alias cannot be used in where clause because where clause is executed before select

to use alias in where clause use derived table

```
SELECT *
FROM ( SELECT empno,ename,sal,
          DENSE_RANK() OVER (ORDER BY sal DESC) as rnk
      FROM emp ) as e
WHERE rnk <= 5
```

=> display top 3 max salaries ?

METHOD 1 :- (using top clause)

```
SELECT DISTINCT TOP 3 sal
FROM emp
ORDER BY sal DESC
```

METHOD 2 :- (using co-related sub-query)

```
SELECT DISTINCT a.sal
FROM emp as a
WHERE 3 > (SELECT COUNT(DISTINCT b.sal)
           FROM emp as b
           WHERE a.sal < b.sal)

ORDER BY sal DESC
```

METHOD 3 :- (using dense_rank & derived table)

```
SELECT DISTINCT sal
FROM ( SELECT sal,
              DENSE_RANK() OVER (ORDER BY sal DESC) as rnk
        FROM emp ) as e
WHERE rnk <= 3
ORDER BY sal DESC
```

=> display 3rd max salary ?

```
SELECT DISTINCT sal
FROM ( SELECT sal,
              DENSE_RANK() OVER (ORDER BY sal DESC) as rnk
        FROM emp ) as e
WHERE rnk = 3
ORDER BY sal DESC
```

Ex 2 :-

=> display first 5 rows from emp table ?

```
SELECT *
FROM (SELECT empno,ename,sal ,
              ROW_NUMBER() OVER (ORDER BY empno ASC) as rno
        FROM emp ) as e
WHERE rno<=5
```

WHERE rno = 5

WHERE rno IN (5,10,15)

WHERE rno BETWEEN 5 AND 10

WHERE rno%2= 0

=> delete first 3 rows from emp table ?

```
DELETE
FROM (SELECT empno,ename,sal ,
              ROW_NUMBER() OVER (ORDER BY empno ASC) as rno
        FROM emp ) as e
WHERE rno<=3 => ERROR
```

=> in derived table outer query cannot be DML and it must be always SELECT

CTE :- (common table expression)
=====

=> using CTE also we can give name to the query output and we can use that name in

another query like SELECT / INSERT / UPDATE / DELETE.

=> using CTE we can simplify complex queries

=> using CTE we can implement step by step process to resolve complex query

```
WITH <CTE1>
AS
  (SELECT statement) ,
<CTE2>
AS
  (SELECT statement)
-----

SELECT / INSERT / UPDATE / DELETE
```

Ex 1 :-

=> display first 5 rows from emp table ?

```
WITH E
AS
  (
    SELECT empno,ename,sal,
           ROW_NUMBER() OVER (ORDER BY empno ASC) as rno
    FROM emp
  )

SELECT * FROM E WHERE rno <= 5
```

=> delete first 3 rows from emp ?

```
WITH E
AS
  (
    SELECT empno,ename,sal,
           ROW_NUMBER() OVER (ORDER BY empno ASC) as rno
    FROM emp
  )

DELETE FROM E WHERE rno <= 3
```

Ex 2 :- delete duplicate rows ?

```
emp44
eno  ename sal
1    A      5000
2    B      6000
3    C      7000
1    A      5000  => duplicate row
2    B      6000  => duplicate row
```

STEP 1 :- generate row numbers with in the group of eno,ename,sal

```
SELECT eno,ename,sal ,
       ROW_NUMBER() OVER (PARTITION BY eno,ename,sal
                           ORDER BY eno ASC) as rno
FROM emp44
```

```
1    A      5000  1
1    A      5000  2

2    B      6000  1
2    B      6000  2
```

```
SELECT COUNTRY,COUNT(*) AS PLAYED
FROM ( SELECT TEAMA AS COUNTRY FROM CRICKET
      UNION ALL
      SELECT TEAMB AS COUNTRY FROM CRICKET
      ) AS T
GROUP BY COUNTRY
```

13-may-26

SUB-QUERIES IN SELECT clause :-
=====

=> sub-queries in SELECT clause are called scalar sub-queries.

```
SELECT (SELECT stmt) , (SELECT stmt) , -----
FROM tablename
WHERE cond
```

=> sub-query output acts like a column for outer query

Ex 1 :-

```
SELECT (SELECT COUNT(*) FROM emp) as emp ,
       (SELECT COUNT(*) FROM dept) as dept
```

```
emp dept
11  4
```

Ex 2 :-

=> display dept wise total salary ?

```
SELECT deptno,SUM(sal) as dept_totsal
FROM emp
GROUP BY deptno
```

```
10      8750.00
20     10075.00
30     6550.00
```

=> display DEPTNO DEPT_TOTSAL TOTSAL ?

```
SELECT deptno,SUM(sal) as dept_totsal,
       (SELECT SUM(sal) FROM emp) as totsalsal
FROM emp
GROUP BY deptno
```

```
10  8750.00    25375.00
20 10075.00    25375.00
30  6550.00    25375.00
```

=> display DEPTNO DEPT_TOTSAL TOTSAL PCT ?

PCT = (DEPT_TOTSAL / TOTSAL) * 100

```
SELECT deptno,SUM(sal) as dept_totsal,
       (SELECT SUM(sal) FROM emp) as totsalsal,
       (SUM(sal) / (SELECT SUM(sal) FROM emp))*100 as pct
FROM emp
GROUP BY deptno
```

=> display DNAME DEPT_TOTSAL TOTSAL PCT ?

```
SELECT d.dname,SUM(e.sal) as dept_totsal,
       (SELECT SUM(sal) FROM emp) as totsalsal,
       (SUM(e.sal) / (SELECT SUM(sal) FROM emp))*100 as
pct
```

```
FROM emp as e INNER JOIN dept as d
ON e.deptno = d.deptno
GROUP BY d.dname
```

Ex 3 :-

```
SALES
dateid      prodid  custid  qty  amt
2026-05-12  100    10     1    2000
```

```
PRODUCTS
prodid  pname price sub-category  category  brand
100     kkk      2000      mobile    electronics
```

xxxx

output :-

```

CATEGORY      CAT_TOTAL_AMOUNT      TOTAL_AMOUNT      PCT
ELECTRONICS    120CR                      800CR
??
```

=====

PIVOT operator :-

=====

=> operator used to convert rows into columns
=> operator used to display data in matrix form

```
SELECT *
FROM (SELECT required data) as <alias>
PIVOT
(
    aggr-expr FOR colname IN (v1,v2,v3,---)
) AS <PIVOT-TBL-NAME>
ORDER BY <colname> ASC/DESC
```

Ex 1 :-

```
SELECT *
FROM (SELECT deptno,job,sal FROM emp) as e
PIVOT
(
    SUM(sal) FOR deptno IN ([10],[20],[30])
) AS dept_job_totsal
```

job	10	20	30
ANALYST		NULL	6000.00
CLERK	1300.00	1100.00	950.00
MANAGER	2450.00	2975.00	2850.00
PRESIDENT	5000.00	NULL	NULL
SALESMAN	NULL	NULL	2750.00

Ex 2 :-

	1	2	3	4
1980	?	?	?	?
1981	?	?	?	?
1982				
1983				


```

SELECT *
FROM ( SELECT DATEPART(yy,hiredate) as year,
              DATEPART(qq,hiredate) as qrt,
              empno
        FROM emp
      ) as e
PIVOT
(
    COUNT(empno) FOR qrt IN ([1],[2],[3],[4])
) AS year_qrt_count

```

Ex 3 :-

STUDENT			
sno	sname	subject	marks
1	A	MAT	80
1	A	PHY	60
1	A	CHE	50
2	B	MAT	70
2	B	PHY	40
2	B	CHE	50

output :-

SNO	SNAME	MAT	PHY	CHE
1	A	80	60	50
2	B	70	40	50

=====

14-may-26

Database Transactions :-
=====

=> a transaction is a unit of work that contains one or more dmls and must be saved as a whole or must be cancelled as a whole.

ex :- money transfer

	acct1		acct2
	-----1000-----		
	update1		update2
	(bal = bal - 1000)		(bal =
bal + 1000)			
	successful	failed	invalid
	failed	successful	invalid
	successful	successful	valid
	failed	failed	valid

=> every txn must guarantee a property called atomocity i.e. all or none i. e. if txn contains multiple operations , if all operations are successfully completed then then it must be saved , if one of the operation fails then entire txn

must be cancelled

=> the following commands provided by SQL SERVER to handle transactions are called

TCL commands

1	commit	=>	to save txn
2	rollback	=>	to cancel txn
3	save transaction	=>	to cancel part of the txn

=> every txn has a begin point and an end point

=> a txn begins implicitly when user submits dml command and ends implicitly with commit

update => a txn begins
update
commit

=> user can also start txn by executing "begin transaction" command and end txn using commit / rollback

ex 1 :-

```
create tabl a(a int)
insert into a values(10)
insert into a values(20)
insert into a values(30)
insert into a values(40)
rollback                                => ERROR
```

create table,insert => implicitly committed

SELECT * FROM a

10
20
30
40

Ex 2 :-

```
create table a(a int)          /* implicitly committed */
BEGIN TRANSACTION             /* txn starts T1 */
insert into a values(10)
insert into a values(20)
insert into a values(30)
insert into a values(40)
ROLLBACK                      /* txn ends */
```

=> if txn ends with ROLLBACK then it is called aborted txn and operations are cancelled

Ex 3 :-

```
create table a(a int)
BEGIN TRANSACTION
insert into a values(10)
insert into a values(20)
insert into a values(30)
insert into a values(40)
COMMIT
```

=> if txn ends with commit then it is called successful txn and operations are saved

Ex 4 :-

```
create table a(a int)                                /* implicitly committed */
BEGIN TRANSACTION                                  /* txn begins T1 */
insert into a values(10)
insert into a values(20)
COMMIT                                              /* txn ends COMMIT */
insert into a values(30)                          /* implicitly committed */
insert into a values(40)                          /* implicitly committed */
ROLLBACK                                           => ERROR => trying to end txn without
starting
```

SAVE TRANSACTION :-
=====

=> we can declare save transaction and we can rollback upto the save transaction
=> using save transaction we can cancel part of the transaction

ex :-

```
create table a(a int)
BEGIN TRANSACTION
insert into a values(10)
insert into a values(20)
SAVE TRANSACTION st1
insert into a values(30)
insert into a values(40)
SAVE TRANSACTION st2
insert into a values(50)
insert into a values(60)
ROLLBACK TRANSACTION st1
```

SELECT * FROM a

10
20

=====

DB Security :-
=====

- 1 LOGINS => provides security at server level
- 2 USERS => provides security at db level
- 3 PRIVILEGES => provides security at table level
- 4 VIEWS => provides security at row & col level

server (logins)
 database (users)
 table (privileges)
 rows & cols (views)

SERVER
 SA
 NARESH

DB730
 DBO (SA)
 emp
 dept
 student
 customers

VIJAY (NARESH)

15-may-26

DB Objects :-
=====

- 1 TABLES
- 2 VIEWS
- 3 SYNONYMS
- 4 SEQUENCES
- 5 INDEXES

VIEWS :-
=====

=> a view is a subset of a table i.e. part of a table
=> a view is a virtual table i.e. because it doesn't store data and doesn't occupy memory
=> view always derives data from base table
=> a view is a representation of a query

purpose :-
=====

- 1 to provide security
- 2 to reduce complexity

Types of views :-
=====

- 1 simple views
- 2 complex views

simple views :-
=====

=> a view said to be simple if it is based on one table

```
CREATE VIEW <name>
AS
SELECT statement
```

Ex :-

```
CREAE VIEW V1
AS
SELECT empno,ename,job,hiredate,deptno FROM emp
```

=> sql server creates view "V1" and stores query but not query output i.e. data

```
SELECT * FROM V1
```

=> sql server executes the above query as follows

```
SELECT * FROM (SELECT empno,ename,job,hiredate,deptno FROM emp)
```

Granting permissions on view to user :-
=====

DBO :-
=====

```
GRANT SELECT,INSERT,UPDATE,DELETE ON V1 TO VIJAY
```

```
VIJAY :-  
=====
```

```
1  SELECT * FROM V1  
  
2  UPDATE V1 SET JOB='ANALYST' WHERE EMPNO=7566  
  
2  UPDATE V1 SET SAL = 3000 WHERE EMPNO=7566 => ERROR  
  
4  INSERT INTO V1 VALUES(999, 'KKK', 'CLERK', GETDATE(), 20)
```

```
ROW LEVEL SECURITY :-  
=====
```

```
DBO :-  
=====
```

```
CREATE VIEW V2  
AS  
SELECT empno,ename,job,deptno  
FROM emp  
WHERE deptno = 20
```

```
GRANT SELECT,INSERT, UPDATE,DELETE ON V2 TO VIJAY
```

```
VIJAY :-  
=====
```

```
1  SELECT * FROM V2
```

```
complex views :-  
=====
```

=> a view said to be complex view

```
1  if based on multiple tables (joins)  
2  if query performs group by  
                                     having  
                                     aggregate  
                                     set operations  
                                     sub-queries
```

=> with the help of views a complex query can be converted into simple query

Ex 1 :-

EMPLOYEES	DEPARTMENTS	LOCATIONS	COUNTRIES
employee_id	department_id		location_id
country_id			
first_name	department_name		city
country_name			
last_name	location_id		state
salary			
department_id			country_id

```
CREATE VIEW CV1  
AS  
SELECT e.employee_id,  
       e.first_name,  
       e.salary,  
       d.department_name,
```

```

        l.city,l.state,
        c.country_name
FROM employees as e
INNER JOIN departments as d
ON e.department_id = d.department_id
INNER JOIN locations as l
ON d.location_id = l.location_id
INNER JOIN countries as c
ON l.country_id = c.country_id

```

=> after creating view then when we want employee details with dept, locations and countries details then write the simple query

```
SELECT * FROM CV1
```

=> difference between simple and complex views ?

	SIMPLE	COMPLEX
1	based on single table	based on multiple tables
2	query performs simple operations like where	query performs complex operations like joins,group by ,aggregate etc
3	allows dmls	doesn't allow dmls

=> list of tables ?

```

SELECT *
FROM INFORMATION_SCHEMA.TABLES
WHERE TABLE_TYPE='BASE TABLE'

```

```

INFORMATION_SCHEMA.TABLES
-----
          schema                      table

```

Dropping :-
=====

```
DROP VIEW V1
```

if base table is dropped then what about views created on base table ?

ANS :- views are not dropped but cannot be queried

schemas :-
=====

```

information_schema
sys
dbo
guest

```

16-may-26

synonyms :-
=====

=> a synonym is another name or alterantive name for a table/view

=> if tablename is lengthy then we can give a simple and short name to the table called synonym and after creating synonym instead of using table

name

use synonym name in SELECT / INSERT / UPDATE / DELETE queries.

CREATE SYNONYM <name> FOR <tablename>

Ex :- CREATE SYNONYM E FOR EMP

Using synonym :-

=====

- 1 SELECT * FROM E
- 2 UPDATE E SET SAL=2000 WHERE EMPNO = 999

Question :-

=====

- 1 CREATE SYNONYM E FOR EMP
- 2 SELECT * FROM EMP AS E
- 3 SP_RENAME 'EMP','E' => changes tablename from emp to e

difference between synonym and alias ?

synonym	alias
1 stored in db	not stored in db
2 permanent	not permanent
3 scope of the synonym is upto the db	scope of the alias is upto the query

=> list of synonyms ?

SELECT name,base_object_name FROM sys.synonyms

E EMP

Dropping :-

=====

DROP SYNONYM E

if base table is dropped what about synonym created on base table ?

ans :- synonyms are not dropped but cannot be queried

Question :-

DROP TABLE products;

What is the implication of this command? (Choose all that apply.)

- A. All data along with the table structure is deleted. T
- B. The pending transaction in the session is committed. T
- C. All views and synonyms will remain but they are invalidated. T
- D. All data in the table are deleted but the table structure will remain. F

SEQUENCE :-

=====

=> a sequence is also a db object created to generate sequence numbers
mainly used for primary key fields

=> sequences are created to auto increment column values

```

CREATE SEQUENCE <name>
[start with <value.>]
[increment by <value>]
[maxvalue <value>]
[minvalue <value>]
[cycle]
[cache <size>]

```

Ex 1 :-

```

CREATE SEQUENCE S1
START WITH 1
INCREMENT BY 1
MAXVALUE 5

```

using sequence :-

```

CREATE TABLE stud(sid INT primary key,sname VARCHAR(10))

```

use sequence S1 to generate values for sid for every new student

```

INSERT INTO stud VALUES(next value for s1 , 'A')
INSERT INTO stud VALUES(next value for s1 , 'B')
INSERT INTO stud VALUES(next value for s1 , 'C')
INSERT INTO stud VALUES(next value for s1 , 'D')
INSERT INTO stud VALUES(next value for s1 , 'E')
INSERT INTO stud VALUES(next value for s1 , 'F')    => ERROR

```

```

SELECT * FROM stud

```

SID	SNAME
1	A
2	B
3	C
4	D
5	E

Ex 2 :-

```

CREATE SEQUENCE S2
START WITH 100
INCREMENT BY 1
MAXVALUE 999

```

=> use sequence S2 to generate value for empno ?

```

UPDATE emp SET empno = next value for s2

```

Ex 3 :-

BILL		
BILLNO	BDATE	AMT
DM/0526/1		
DM/0526/2		

```

CREATE TABLE BILL
(
BILLNO    VARCHAR(20) PRIMARY KEY,
BDATE     DATETIME,
AMT       MONEY
)

```



```
CREATE SEQUENCE S3
START WITH 1
INCREMENT BY 1
MAXVALUE 99999
```

use sequence s3 to generate billno ?

```
INSERT INTO BILL
VALUES('DM/' +
      FORMAT(getdate(), 'MMyy') + '/' +
      CAST(next value for s3 AS VARCHAR) , getdate() , 2000)
```

```
SELECT * FROM bill
```

```
DM/0526/1    2026-05-16 08:40:44.860 2000.00
DM/0526/2    2026-05-16 08:40:51.517 3000.00
DM/0526/3    2026-05-16 08:40:57.600 5000.00
```

How to restart the sequence ?

- 1 manually
- 2 automatically

manually :-
=====

```
CREATE SEQUENCE S10
START WITH 1
INCREMENT BY 1
MAXVALUE 5
```

=> after reaching max then to restart execute the following command

```
ALTER SEQUENCE S10 RESTART WITH 1
```

Automatically :-
=====

```
CREATE SEQUENCE S11
START WITH 1
INCREMENT BY 1
MAXVALUE 5
MINVALUE 1
CYCLE
```

=> list of sequences created by user ?

```
SELECT * FROM information_schema.sequences
```

Dropping :-
=====

```
DROP SEQUENCE S1
```

18-may-26

Indexes :-
=====

=> index is also a db object created to improve the performance of data accessing.

=> index makes data accessing faster.

=> index in db is similar to index in textbook , In textbook using index a

particular

topic can be located fastly but in db using index a particular row/rows can be

located fastly.

=> indexes created on columns and that column is called index key

=> indexes created on columns

1 which are frequently used in where clause

2 which are used in join operation

Types of Indexes :-

=====

1 Non Clustered

simple index

composite index

unique index

2 Clustered

Simple Non Clustered :-

=====

=> if index created on single column then it is called simple index

CREATE INDEX <name> ON <tablename>(<colname>)

Ex :- CREATE INDEX I1 ON emp(sal)

=> when above command is executed sql server creates a structure called BTREE (binary tree)

```
EMP          3000
sal
5000
1000          2000          4000
3000
2000      1000 *          2500 *          4000 *
5000 *
1500          1500 *          3000 *, *
4000      2000 *
3000
2500
```

=> when we submit a query , sql server uses following methods to locate the records

1 TABLE SCAN

2 INDEX SCAN

=> in table scan sql server scans complete table i.e. each and every row.

=> in index scan on avg sql server scans half of the table , so index scan is faster than table scan.

SELECT * FROM emp WHERE sal = 3000 (index scan)

SELECT * FROM emp WHERE sal >= 3000 (index scan)

SELECT * FROM emp WHERE sal <= 3000 (index scan)

SELECT * FROM emp WHERE ename='blake' (table scan)

SELECT * FROM emp (table scan)

scan)

SELECT * FROM emp WHERE sal <> 3000 (table scan)

composite index :-

=====

=> if index created on combination of columns then it is called composite index

```
CREATE INDEX I2 ON EMP(DEPTNO, JOB)
```

20

10

30

```
10 CLERK *          20 ANALYST *,*      30 CLER *
10 MGR *            20 CLERK *,*        30 MGR *
                20 MGR *  30 SALESMAN *,*,*,*
```

=> sql server uses above index when where cond based on leading column of the index i.e. deptno

```
SELECT * FROM emp WHERE deptno = 20 (index scan)
SELECT * FROM emp WHERE deptno=20 AND job='CLERK' (index scan)
SELECT * FROM emp WHERE job='CLERK' (table scan)
```

```
select * from emp where deptno = 20
SELECT * FROM emp WHERE deptno=20 and job='clerk'
```

UNIQUE index :-

=====

=> unique index doesn't allow duplicate values into the column on which index is created

```
CREATE UNIQUE INDEX I3 ON emp(ename)
```

K

G

Q

```
ADAMS *      JAMES *      MARTIN *      SCOTT *
ALLEN *      JONES *      MILLER *      SMITH *
BLAKE *      KING *
```

```
1 SELECT * FROM emp WHERE ename = 'BLAKE'
```

```
2 INSERT INTO emp(empno,ename,sal) VALUES(999,'BLAKE',2000) => ERROR
```

what are the different methods to enforce uniqueness ?

- 1 declare primary key / unique constraint
- 2 create unique index

=> primary key / unique columns are automatically indexed by sql server and sql server creates a unique index on primary key / unique columns and unique index doesn't allow duplicates , so primary key / unique also doesn't allow duplicates.

CLUSTERED indexes :-

=====

=> a non clustered index stores pointers to actual records where as clustered index stores actual records

=> In non clustered , index and table are two separate objects where as in clustered index

index and table are not two separate objects and they are one object.

ex :-

step 1 :- create table

```
CREATE TABLE cust(cid INT, cname VARCHAR(10))
```

step 2 :- create clustered index

```
CREATE CLUSTERED INDEX I10 ON CUST(cid)
```

step 3 :- insert data into table

```
INSERT INTO cust VALUES(10,'A')
INSERT INTO cust VALUES(80,'B')
INSERT INTO cust VALUES(40,'C')
INSERT INTO cust VALUES(60,'D')
```

		50					
	30		70				
10	A	40	C	60	D	80	B

1 SELECT * FROM CUST

for the above query sql server goes to clustered index and access all leaf nodes
from left to right

	10	A
40	C	
60	D	
80	B	

2 SELECT * FROM CUST WHERE CID = 60

for the above query sql server goes to clustered index and finds the entry cid=60
and after finding then return the row from index

=> a clustered index is created on primary key columns and a non clustered index is created on unique column

difference between a non clustered and clustered index ?

NON-CLUSTERED

CLUSTERED

1 stores addresses of actual records

stores actual records

2 index and table are two separate objects

index and table are not separate objects they are one

3 needs extra storage

doesn't need extra storage

4 requires two lookups one in index and one in table
one lookup i.e. in index

needs only

5 a non clustered index implicitly created on unique col implicitly
created on primary key column

6 max 999 non clustered indexes allowed per table only one clustered
index allowed per table

list of indexes ?

SP_HELPINDEX CUST

Dropping :-

=====

DROP INDEX EMP.I1

=> if we drop table what about indexes created on table ?

ANS :- indexes are also dropped

SERVER

DATABASE

TABLES

ROWS & COLS

CONSTRAINTS

INDEXES

TRIGGERS

VIEWS

SYNONYMS

SEQUENCES

SQL

COMMANDS	CLAUSES	OPERATION	FUNCTIONS	OBJECTS
DDL	WHERE	FILTERING	CHAR	TABLES
DML	ORDER	SORTING	DATE	VIEWS
DRL	DISTINCT	DUPLICATES	NUMERIC	SYNONYMS
TCL	TOP	TOP N ROWS	CONVERSION	SEQUENCES
DCL	GROUP BY	GROUPING	SPECIAL	INDEXES
	HAVING	FILTER	GROUPS	ANALYTICAL
	ON	JOIN	AGGREGATE	
	WITH	SUB-QUERIES		
		SET OPERATIONS		
		PIVOTING		

=====

20-may-26

T-SQL PROGRAMMING

=====

- 1 Basic programming
- 2 conditional statements
- 3 loops
- 4 cursors
- 5 error handling
- 6 stored procedures
- 7 functions
- 8 triggers
- 9 dynamic sql

Features :-

=====

1 improves performance :-
=====

=> In t-sql , sql commands can be grouped into one block and we submit that block to sql server , so no of requests and response between user and sql server are reduced and performance is improved.

2 supports conditional statement :-
=====

=> t-sql supports conditional statements like if-else and with the help of these statements we can execute sql queries based on conditions.

3 supports loops :-
=====

=> t-sql supports looping statements like while and with the help of loops we can execute sql queries repeatedly multiple times.

4 supports error handling :-
=====

=> In t-sql , if any statement causes error then we can handle that error and we can replace system generate message with our own simple message.

5 supports reusability :-
=====

=> t-sql programs can be stored in db, so applications which are connected to db can reuse t-sql programs.

=> T-SQL blocks are 2 types

- 1 Anonymous Blocks
- 2 Named Blocks
 - stored procedure
 - functions
 - triggers

Anonymous Blocks :-
=====

=> a t-sql block without name is called anonymous block
=> the following statements are used in T-SQL blocks

- 1 DECLARE
- 2 SET
- 3 PRINT

DECLARE :-
=====

=> statement used to declare variables

DECLARE @varname datatype(size)

Ex :-

```
DECLARE @a int
DECLARE @s varchar(10)
DECLARE @d date

DECLARE @a int,@s varchar(10),@d date
```

SET :-
=====

=> statement used to assign value to variable

SET @varname = value

Ex :-
SET @a = 100
SET @s = 'abc'
SET @d = getdate()

PRINT :-
=====

=> statement used to print values

PRINT message / @varname

Ex :-
PRINT 'HELLO'
PRINT @a

=> wap to add two numbers ?

```
DECLARE @a INT,@b INT,@c INT
SET @a=100
SET @b=200
SET @c = @a+@b
PRINT @c
```

=> wap to input date and print day of the week ?

```
DECLARE @d DATE
SET @d = '1947-08-15'
PRINT DATENAME(DW,@d)
```

=> wap to input name and print first name,middle name,last name ?

input :- sachin ramesh tendulkar

output :-

```
First Name = sachin
Middle Name = ramesh
Last Name = tendulkar
```

difference between SUBSTR & REGEXP_SUBSTR ?

```
SUBSTR(string,start,no of chars)
REGEXP_SUBSTR(string,pattern,[start,occurance,case] )
```

In substr extracting part of the string is based on start and length
In regexp_substr extracting part of the string is based on pattern

```
DECLARE @n VARCHAR(60),@f VARCHAR(20)
DECLARE @m VARCHAR(20),@l VARCHAR(20)
SET @n = 'sachin ramesh tendulkar'
SET @f = REGEXP_SUBSTR(@n,'[a-z]+',1,1)
SET @m = REGEXP_SUBSTR(@n,'[a-z]+',1,2)
SET @l = REGEXP_SUBSTR(@n,'[a-z]+',1,3)
PRINT ' First Name = ' + @f
PRINT ' Middle Name = ' + @m
PRINT ' Last Name = ' + @l
```

DB programming in T-SQL :-

=====

=> from t-sql to work with db execute sql commands from t-sql program
=> the following commands are executed from t-sql programs

- 1 DML (insert,update,delete,merge)
- 2 DRL (select)
- 3 TCL (commit,rollback,save transaction)

SELECT stmt :-

=====

```
SELECT  @var1 = col1 ,
        @var2 = col2 ,
        -----
```

```
FROM tablename
[WHERE cond]
```

Ex :-

```
SELECT  @n = ename , @s = sal FROM emp WHERE empno = 105
```

=> wap to input empno and print name & salary ?

```
DECLARE @n VARCHAR(20),@s MONEY
SELECT  @n = ename , @s = sal FROM emp WHERE empno = 105
PRINT  @n + ' ' + CAST(@s AS VARCHAR)
```

21-may-26

=> wap to input empno and calculate and print experience of the employee ?

```
DECLARE @eno INT ,@hire DATE,@expr INT
SET @eno = 110
SELECT  @hire = hiredate FROM emp WHERE empno = @eno
SET @expr = DATEDIFF(yy,@hire,getdate())
PRINT 'Experience = ' + CAST(@expr AS VARCHAR) + ' Years'
```

output :-

Experience = 43 Years

Conditional Statements :-

=====

- 1 IF-ELSE
- 2 MULTI IF
- 3 NESTED IF

IF-ELSE :-

=====

```
IF COND
BEGIN
    statements
END
ELSE
BEGIN
    statements
END
```


MULTI-IF :-

=====

```
IF COND1
BEGIN
    statements
END
ELSE IF COND2
BEGIN
    statements
END
ELSE IF COND3
BEGIN
    statements
END
ELSE
BEGIN
    statements
END
```

NESTED IF :-

=====

```
IF COND
BEGIN
    IF COND
    BEGIN
        statements
    END
    ELSE
    BEGIN
        statements
    END
END
ELSE
BEGIN
    statements
END
```

=> wap to input empno and increment sal by specific amount
after increment if sal exceeds max salary then cancel that increment ?

```
DECLARE @eno INT ,@amt MONEY,@sal MONEY,@maxsal MONEY
SET @eno = 107
SET @amt = 2500
SELECT @maxsal = MAX(sal) FROM emp
BEGIN TRANSACTION
UPDATE emp SET sal = sal + @amt WHERE empno = @eno
SELECT @sal = sal FROM emp WHERE empno = @eno
IF @sal > @maxsal
    ROLLBACK
ELSE
    COMMIT
```

=> wap to input empno and increment sal as follows ?

if job=CLERK	incr sal by 10%	
SALESMAN		15%
MANAGER		20%
others		5%

```
DECLARE @eno INT,@job VARCHAR(10),@pct INT
SET @eno = 104
SELECT @job = job FROM emp WHERE empno = @eno
```

```

IF @job='CLERK'
    SET @pct = 10
ELSE IF @job='SALESMAN'
    SET @pct = 15
ELSE IF @job='MANAGER'
    SET @pct = 20
ELSE
    SET @pct = 5
UPDATE emp SET sal = sal + (sal*@pct /100) WHERE empno = @eno

```

=> wap to process bank transaction (W/D) ?

```

ACCOUNTS
accno    actype    bal
100 S    10000
101 S    20000

```

```

TRANSACTIONS
trid      ttype tdate tamt  accno

```

```

CREATE TABLE ACCOUNTS
(
    ACCNO INT PRIMARY KEY,
    ACTYPE VARCHAR(1),
    BAL MONEY
)

```

```

INSERT INTO ACCOUNTS VALUES(100, 'S', 10000), (101, 'S', 20000)

```

```

CREATE TABLE TRANSACTIONS
(
    TRID INT ,
    TTYPE VARCHAR(1),
    TDATE DATETIME,
    TAMT MONEY,
    ACCNO INT REFERENCES ACCOUNTS(ACCNO)
)

```

```

DECLARE @acno INT , @type VARCHAR(1), @tamt MONEY, @bal MONEY
SET @acno = 100
SET @type='W'
SET @amt = 1000
IF @type='W'
BEGIN
    SELECT @bal = bal FROM accounts WHERE accno = @acno
    IF @amt > @bal
        PRINT 'insufficient balance'
    ELSE
    BEGIN
        UPDATE accounts SET bal = bal - @amt WHERE accno = @acno
        INSERT INTO transactions VALUES(next value for
s11, 'W', getdate(), @amt, @acno)
    END
END
ELSE IF @type='D'
BEGIN
    UPDATE accounts SET bal = bal + @amt WHERE accno = @acno
    INSERT INTO transactions VALUES(next value for
s11, 'D', getdate(), @amt, @acno)
END
ELSE
    PRINT 'invalid transaction type'

```

=> wap to input sno and calculate total,avg,result and insert into result table ?

student				
sid	sname	s1	s2	s3
1	A	80	90	70
2	B	30	50	60

result			
sno	total	avg	result

```
DECLARE @sid INT , @s1 INT,@s2 INT,@s3 INT
DECLARE @total INT,@avg DECIMAL(5,2),@result VARCHAR(4)
SET @sid = 1
SELECT @s1 = s1,@s2 = s2,@s3 = s3 FROM student WHERE sid = @sid
SET @total = @s1 + @s2 + @s3
SET @avg = @total/3
IF @s1>=35 AND @s2>=35 AND @s3>=35
    SET @result = 'pass'
ELSE
    SET @result = 'fail'
INSERT INTO result VALUES(@sid,@total,@avg,@result)
```

WHILE loop :-
=====

```
WHILE(cond)
BEGIN
    statements
END
```

if cond = true loop continues
if cond = false loop terminates

Ex :-

=> wap to print numbers from 1 to 20 ?

```
DECLARE @x INT = 1
WHILE(@x<=20)
BEGIN
    PRINT @x
    SET @x = @x + 1
END
```

How to the same in sql :-
=====

=> a function called generate_series is introduced in sql server 2025 ver which is used to generate series

GENERATE_SERIES(start,end,step)

Ex :-

=> waq to print numbers from 1 to 20 ?

```
SELECT *
FROM GENERATE_SERIES(1,20,1)
```

=> waq to print even numbers ?

```
SELECT *
```

```
FROM GENERATE_SERIES(2,20,2)
```

=> wap to print 2026 calendar ?

```
DATE      DAY
2026-01-01 Thursday
2026-01-02 Friday
```

```
2026-12-31 ??
```

```
DECLARE @d1 DATE,@d2 DATE
SET @d1 = '2026-01-01'
SET @d2 = '2026-12-31'
WHILE(@d1 <= @d2)
BEGIN
    PRINT CAST(@d1 AS VARCHAR) + ' ' + DATENAME(DW,@d1)
    SET @d1 = DATEADD(dd,1,@d1)
END
```

How to do it sql ?

```
SELECT
    CAST( DATEADD(DD,VALUE, '2025-12-31') AS DATE) AS DT ,
    DATENAME(DW, CAST( DATEADD(DD,VALUE, '2025-12-31') AS DATE)) AS DAY
FROM GENERATE_SERIES(1,365,1)
```

=> wap to print sundays in 2026 year ?

```
DECLARE @d1 DATE,@d2 DATE
SET @d1 = '2026-01-01'
SET @d2 = '2026-12-31'
WHILE(@d1 <= @d2)
BEGIN
    IF DATENAME(DW,@d1) = 'sunday'
        PRINT CAST(@d1 AS VARCHAR) + ' ' + DATENAME(DW,@d1)
    SET @d1 = DATEADD(dd,1,@d1)
END
```

```
DECLARE @d1 DATE,@d2 DATE
SET @d1 = '2026-01-01'
SET @d2 = '2026-12-31'
/* to get 1st sunday */
WHILE(DATENAME(dw,@d1) <> 'sunday')
BEGIN
    SET @d1 = DATEADD(dd,1,@d1)
END
/* to print sundays */
WHILE(@d1 <= @d2)
BEGIN
    PRINT CAST(@d1 AS VARCHAR) + ' ' + DATENAME(DW,@d1)
    SET @d1 = DATEADD(dd,7,@d1)
END
```

=> wap to insert emi dates into the below table ?

```
house loan :- 40 lacs
first emi date :- 2026-06-05
tenure :- 20 years
```

```
EMI_DATES
SNO DUE_DATE      AMOUNT
1   2026-06-05    ?
2   2026-07-05    ?
```

```
first date :- next month 5th
last date :- first emi date + 20 years
```

=====

23-may-26

CURSORS :-
=====

=> cursors are used to process multiple rows in t-sql programs
=> using cursors we can access row-by-row in t-sql program
=> to use cursor follow below steps

- 1 DECLARE CURSOR
- 2 OPEN CURSOR
- 3 FETCH RECORDS FROM CURSOR
- 4 CLOSE CURSOR
- 5 DEALLOCATE CURSOR

DECLARE CURSOR :-
=====

DECLARE <name> CURSOR FOR SELECT statement

EX :-

DECLARE C1 CURSOR FOR SELECT ename,sal FROM emp

OPENING CURSOR :-
=====

OPEN C1

Ex :-

- 1 sql server executes select query
- 2 data returned by select query is copied to cursor
- 3 c1 points to cursor

FETCHING RECORDS FROM CURSOR :-
=====

=> "FETCH" statement is used to fetch records from cursor

FETCH NEXT FROM <cursor> INTO <variables> ;

Ex :- FETCH NEXT FROM C1 INTO @n,@s

=> a fetch statement fetches one row at a time but to process multiple rows
fetch statement should be executed multiple times , so fetch stmt should
be in a loop

CLOSING CURSOR :-
=====

CLOSE C1

DEALLOCATE CURSOR :-
=====

DEALLOCATE C1

@@FETCH_STATUS :-
=====

=> this returns whether fetch is successful or not

0 => if fetch successful
-1 => if fetch unsuccessful

=> It is useful for loop condition

Ex :-

=> wap to print all employee names and salaries ?

```
DECLARE C1 CURSOR FOR SELECT ename,sal FROM emp
DECLARE @name VARCHAR(10),@sal MONEY
OPEN C1
FETCH NEXT FROM C1 INTO @name,@sal
WHILE(@@FETCH_STATUS=0)
BEGIN
    PRINT @name + ' ' + CAST(@sal as varchar)
    FETCH NEXT FROM C1 INTO @name,@sal
END
CLOSE C1
DEALLOCATE C1
```

=> wap to calculate total sal without using sum ?

```
DECLARE C1 CURSOR FOR SELECT sal FROM emp
DECLARE @sal MONEY,@total MONEY = 0
OPEN C1
FETCH NEXT FROM C1 INTO @sal
WHILE(@@FETCH_STATUS=0)
BEGIN
    SET @total = @total + @sal
    FETCH NEXT FROM C1 INTO @sal
END
CLOSE C1
DEALLOCATE C1
PRINT @total
```

=> wap to print max sal without using max function ?

```
DECLARE C1 CURSOR FOR SELECT sal FROM emp
DECLARE @sal MONEY,@max MONEY
OPEN C1
FETCH NEXT FROM C1 INTO @max
WHILE(@@FETCH_STATUS=0)
BEGIN
    FETCH NEXT FROM C1 INTO @sal
    IF @sal > @max
        SET @max = @sal
END
CLOSE C1
DEALLOCATE C1
PRINT @max
```

```
DECLARE C1 CURSOR FOR SELECT sal FROM emp ORDER BY sal DESC
DECLARE @sal MONEY
OPEN C1
FETCH NEXT FROM C1 INTO @sal
PRINT @sal
```

```
CLOSE C1
DEALLOCATE C1
```

=> wap to print min salary ?

=>

=> wap to calculate all students total, avg, result and insert into result table ?

student				
sid	sname	s1	s2	s3
1	A	80	90	70
2	B	30	50	60

result			
sno	total	avg	result

```
DECLARE C1 CURSOR FOR SELECT sid,s1,s2,s3 FROM student
DECLARE @sid INT , @s1 INT,@s2 INT,@s3 INT
DECLARE @total INT,@avg DECIMAL(5,2),@result VARCHAR(4)
OPEN C1
FETCH NEXT FROM C1 INTO @sid,@s1,@s2,@s3
WHILE(@@FETCH_STATUS=0)
BEGIN
    SET @total = @s1 + @s2 + @s3
    SET @avg = @total/3
    IF @s1>=35 AND @s2>=35 AND @s3>=35
        SET @result = 'pass'
    ELSE
        SET @result = 'fail'
    INSERT INTO result VALUES(@sid,@total,@avg,@result)
    FETCH NEXT FROM C1 INTO @sid,@s1,@s2,@s3
END
CLOSE C1
DEALLOCATE C1
```

student			
sid	s1	s2	s3
1	80	90	70
2	30	50	60

=====

25-may-26

ERROR HANDLING / EXCEPTION HANDLING :-

=====

- 1 syntax errors
- 2 logical errors
- 3 runtime errors

=> errors that are raised during program execution are called runtime errors

```
ex :-    declare @x tinyint
         set @x=1000          => runtime error
         print @x
```

=> if any statement causes runtime error then sql server displays error message and also

continues program execution. To stop program execution and to replace system generated message with our own simple and user friendly message then we need to

handle that
runtime error.

=> To handle runtime errors include a block TRY-----CATCH block

```
BEGIN TRY
    statements    => stmts causes exception
END TRY
BEGIN CATCH
    statements    => stmts handles exception
END CATCH
```

=> if any statement in try block causes exception then control transferred to catch block
and executes the statements in catch block.

Ex 1 :-

```
declare @a tinyint,@b tinyint,@c tinyint
BEGIN TRY
set @a=1000
set @b=0
set @c = @a/@b
print @c
END TRY
BEGIN CATCH
PRINT 'ERROR----TRY AGAIN'
END CATCH
```

Ex 2 :-

```
DECLARE @a tinyint,@b tinyint,@c tinyint
BEGIN TRY
set @a=100
set @b=0
set @c = @a/@b
print @c
END TRY
BEGIN CATCH
    IF ERROR_NUMBER() = 220
        PRINT 'value exceeding limit'
    ELSE IF ERROR_NUMBER() = 8134
        PRINT 'divisor cannot be zero'
END CATCH
```

Ex 3 :-

```
CREATE TABLE emp66
(
    empid INT PRIMARY KEY,
    ename VARCHAR(10) NOT NULL,
    sal MONEY CHECK(sal>=3000)
)
```

wap to insert data into above table ?

```
DECLARE @eno INT,@name VARCHAR(10),@sal MONEY
BEGIN TRY
SET @eno = 101
SET @name='B'
SET @sal = 1000
INSERT INTO emp66 VALUES(@eno,@name,@sal)
END TRY
BEGIN CATCH
```



```

IF ERROR_NUMBER() = 2627
    PRINT 'empno should not be duplicate'
ELSE IF ERROR_NUMBER() = 515
    PRINT 'name should not be null'
ELSE IF ERROR_NUMBER() = 547
    PRINT 'sal >= 3000'
END CATCH

```

RAISERROR() :-
=====

=> It is a built-in procedure used to raise our own errors

RAISERROR(error message,error severity level,state)

severity level => 0 to 25

0â(10 Informational messages
11â(18 Errors
19â(25 Fatal errors

state => 0 to 255 => useful when same error raise at multiple locations

Ex :-

=> wap to input empno and increment sal by specific amount
but sunday updates are not allowed ?

```

DECLARE @eno INT,@amt MONEY
SET @eno = 100
SET @amt = 1000
IF DATENAME(DW,GETDATE()) = 'SUNDAY'
    RAISERROR('sunday not allowed',16,1)
ELSE
    UPDATE emp SET sal = sal + @amt WHERE empno = @eno

```

0 - 49999 => system defined errors
50000 => user defined error

Ex 2 :-

```

CREATE TABLE emp66
(
    empid INT PRIMARY KEY,
    ename VARCHAR(10) NOT NULL,
    sal MONEY CHECK(sal>=3000)
)

```

wap to insert data into above table ?

```

DECLARE @eno INT,@name VARCHAR(10),@sal MONEY
BEGIN TRY
SET @eno = 101
SET @name='B'
SET @sal = 1000
INSERT INTO emp66 VALUES(@eno,@name,@sal)
END TRY
BEGIN CATCH
    IF ERROR_NUMBER() = 2627

```

```

        RAISERROR('empno should not be duplicate',16,1)
ELSE IF ERROR_NUMBER() = 515
    RAISERROR('name should not be null',16,1)
ELSE IF ERROR_NUMBER() = 547
    RAISERROR('sal >= 3000',16,1)
END CATCH

```

26-may-26

Named T-SQL Blocks :-
=====

- 1 procedures
- 2 functions
- 3 triggers

Advantages :-
=====

- 1 modular programming :-

=> with the help of procedures & functions a big T-SQL program can be divided into small modules.

- 2 reusability :-

=> because these programs are created with name , so we can call these programs from another program

- 3 invoked from front-end :-

=====

=> because these programs are stored in db , so these programs can be invoked from front-end applications like c#,asp,python etc

- 4 improves performance :-

=====

=> proc / func improves performance because of one time compilation i.e. if we create a procedure program is compiled and stored in db and whenever we call procedure only execution is repeated but not compilation,
so this improves performance.

PROCEDURES :-
=====

=> a procedure is a named T-SQL block that accepts some input and perform some action on db and may or may not returns a value.

=> procedures are created to perform one or more dml operations on tables

```

CREATE OR ALTER PROCEDURE <name>
parameters if any
AS
    statements

```

parameters :-
=====

=> we can declare parameters and we can pass values to parameters
=> parameters are 2 types

```
1 INPUT (DEFAULT)
2 OUTPUT
```

=> INPUT parameter always receives value from calling program
=> OUTPUT parameter always sends value to calling program

```
C#                                PROCEDURE

A  -----> X (INPUT)

B  <----- Y (OUTPUT)
```

Ex 1 :- procedure without parameter

=> create a procedure to increment all employees sal by 1000 ?

```
CREATE OR ALTER PROCEDURE raise_salary
AS
    UPDATE emp SET sal = sal + 1000
```

procedure created (compiled + stored in db)

Execution :-

```
EXECUTE raise_salary
```

Ex 2 :- procedure with INPUT parameters

=> create a procedure to increment specific employee sal by specific amount ?

```
CREATE OR ALTER PROCEDURE raise_salary
@eno INT ,
@amt MONEY
AS
    UPDATE emp SET sal = sal + @amt WHERE empno = @eno
```

Execution :-

```
EXECUTE raise_salary 100,1000
```

Ex 3 :- procedure with OUTPUT parameter

=> create a procedure to increment specific employee sal by specific amount
and after increment send the updated sal to calling program

```
CREATE OR ALTER PROCEDURE raise_salary
@eno INT,
@amt MONEY,
@newsal MONEY OUTPUT
AS
    UPDATE emp SET sal = sal + @amt WHERE empno = @eno
    SELECT @newsal = sal FROM emp WHERE empno = @eno
```

Execution :-
=====

```
DECLARE @s MONEY
EXECUTE raise_salary 100,1000 ,@s OUTPUT      (positional association)
PRINT @s
```

```

DECLARE @s MONEY
EXECUTE raise_salary @eno=100,@amt=1000 ,@newsal = @s OUTPUT (name
association)
PRINT @s

```

Declaring parameter with default value :-

=====

=> if parameter declared with default value then while calling if we skip value to that parameter then sql server assigns default value.

```

CREATE OR ALTER PROCEDURE raise_salary
@eno INT,
@amt MONEY = 500,
@newsal MONEY OUTPUT
AS
    UPDATE emp SET sal = sal + @amt WHERE empno = @eno
    SELECT @newsal = sal FROM emp WHERE empno = @eno

```

Execution :-

```

DECLARE @s MONEY
EXECUTE raise_salary 100,DEFAULT,@s OUTPUT
PRINT @s

```

```

DECLARE @s MONEY
EXECUTE raise_salary @eno=100,@newsal = @s output
PRINT @s

```

=> create following procedures to implement various bank transactions ?

- 1 opening account
- 2 closing account
- 3 balance enquiry
- 4 money deposit
- 5 money withdrawl
- 6 money transfer

```

ACCOUNTS
accno  actype    bal
100 S    10000
101 S    20000

```

```

TRANSACTIONS
trid    ttype tdate tamt  accno

```

```

CREATE SEQUENCE S10
START WITH 1
INCREMENT BY 1
MAXVALUE 99999

```

```

CREATE TABLE ACCOUNTS
(
ACCNO INT PRIMARY KEY,
ACTYPE VARCHAR(1),
BAL MONEY
)

```

```

INSERT INTO ACCOUNTS VALUES(100, 'S', 10000), (101, 'S', 20000)

```

```

CREATE TABLE TRANSACTIONS

```

```
(
  TRID INT ,
  TTYPE VARCHAR(1),
  TDATE DATETIME,
  TAMT MONEY,
  ACCNO INT REFERENCES ACCOUNTS(ACCNO)
)
```

27-may-26

=> create a procedure for money withdrawl ?

```
CREATE OR ALTER PROCEDURE DEBIT
@acno INT ,
@amt MONEY,
@newbal MONEY OUTPUT
AS
    DECLARE @bal MONEY
    SELECT @bal = bal FROM accounts WHERE accno = @acno
    IF @amt > @bal
        RAISERROR('insufficient balance',16,1)
    ELSE
        BEGIN
            UPDATE accounts SET bal = bal - @amt WHERE accno = @acno
            INSERT INTO transactions VALUES(next value for
s10, 'W',getdate(),@amt,@acno)
            SELECT @newbal = bal FROM accounts WHERE accno = @acno
        END
```

Execution :-

=====

```
DECLARE @b MONEY
EXEC DEBIT 100,10000,@b OUTPUT
PRINT @b
```

USER DEFINE FUNCTIONS :-

=====

=> when predefine functions not meeting your requirements then we create our own

functions called user define functions.

=> a function is also a named T-SQL block that accepts some input performs some

calculation and must return a value.

=> functtions are created

- 1 for calculations
- 2 to fetch data from db

=> functions are 2 types

- 1 scalar valued functions (svf)
- 2 table valued functions (tvf)

scalar valued functions :-

=====

=> these functions returns one value

=> return type must be scalar types like int,varchar,date,money etc

=> return expr must be a scalar variable

```

CREATE OR ALTER
    FUNCTION <name> (parameters) RETURNS <type>
AS
BEGIN
    statements
    RETURN <expr>
END

```

Ex 1 :-

=> create a function that accepts empno and returns experience ?

```

CREATE OR ALTER
    FUNCTION getExpr(@eno INT) RETURNS INT
AS
BEGIN
    DECLARE @hire DATE ,@expr INT
    SELECT @hire = hiredate FROM emp WHERE empno = @eno
    SET @expr = DATEDIFF(yy,@hire,getdate())
    RETURN @expr
END

```

Execution :-

- 1 sql commands
- 2 another program
- 3 .net application

```
SELECT dbo.getexpr(100)  => 46
```

Ex 2 :-

ORDERS			PRODUCTS		
ordid	prodid	qty		prodid	pname price
1000	100	2	100	A	1000
1000	101	3	101	B	500
1000	102	1	102	C	2000
1001	100	2			

=> create a function to calculate total amount of particular order ?

input ordid = 1000

output amount = 5500

```

CREATE OR ALTER
    FUNCTION getOrdAmt(@d INT) RETURNS MONEY
AS
BEGIN
    DECLARE C1 CURSOR FOR SELECT  o.prodid,o.qty,p.price
                                FROM orders as o INNER
JOIN products as p
                                ON o.prodid = p.prodid
                                WHERE o.ordid = @d

    DECLARE @pid INT,@qty INT,@price MONEY,@total MONEY = 0
    OPEN C1
    FETCH NEXT FROM C1 INTO  @pid,@qty,@price
    WHILE(@@FETCH_STATUS=0)
    BEGIN
        SET @total = @total + (@qty * @price)
        FETCH NEXT FROM C1 INTO  @pid,@qty,@price
    END
    CLOSE C1

```

```

        DEALLOCATE C1
        RETURN @total
END

```

```

C1
  prodid  qty  price
100  2    1000
101  3     500
102  1    2000

```

Execution :-

```

SELECT dbo.getOrdAmt(1000) as order_amount      => 5500
SELECT dbo.getOrdAmt(1001) as order_amount      => 2000

```

28-may-26

TABLE VALUED FUNCTIONS :-
=====

```

=> table valued functions returns rows
=> return type must be TABLE
=> return expr must be select stmt
=> TVF are invoked in FROM clause

```

```

CREATE OR ALTER
    FUNCTION <name>(parameters) RETURNS TABLE
AS
    RETURN (SELECT statement)

```

Ex 1 :-

=> create a function that accepts deptno and returns employee list working for that dept ?

```

CREATE OR ALTER
    FUNCTION getEmpList(@dno INT) RETURNS TABLE
AS
    RETURN (SELECT * FROM emp WHERE deptno = @dno)

```

Execution :-

```

SELECT * FROM dbo.getEmpList(30)

```

Ex 2 :-

=> create a function that returns Top N employees list based on sal ?

```

CREATE OR ALTER FUNCTION getTopN(@n INT) RETURNS TABLE
AS
    RETURN ( WITH E
              AS(
                SELECT empno,ename,sal,
                      DENSE_RANK() OVER (ORDER BY
sal DESC) as rnk
              FROM emp
            )
            SELECT * FROM E WHERE rnk <= @n
          )

```

Execution :-

```
SELECT * FROM dbo.getTopN(3)
```

=> create following procedures & functions to implement various bank transactions ?

- 1 opening account (proc)
- 2 closing account (proc)
- 3 balance enquiry (svf)
- 4 money deposit (proc)
- 5 money withdrawl (proc)
- 6 money transfer (proc)
- 7 statement between two given dates (tvf)
- 8 latest N transactions of particular customer (tvf)

```
ACCOUNTS
accno  actype    bal
100 S    10000
101 S    20000
```

```
TRANSACTIONS
trid    ttype tdate tamt  accno
```

```
CREATE SEQUENCE S10
START WITH 1
INCREMENT BY 1
MAXVALUE 99999
```

```
CREATE TABLE ACCOUNTS
(
ACCNO INT PRIMARY KEY,
ACTYPE VARCHAR(1),
BAL MONEY
)
```

```
INSERT INTO ACCOUNTS VALUES(100, 'S', 10000), (101, 'S', 20000)
```

```
CREATE TABLE TRANSACTIONS
(
TRID INT ,
TTYPE VARCHAR(1),
TDATE DATETIME,
TAMT MONEY,
ACCNO INT REFERENCES ACCOUNTS(ACCNO)
)
```

=> difference between procedures and functions ?

PROCEDURES	FUNCTIONS
1 may or may not returns a value	must return a value
2 can return multiple values	always returns one value
3 returns values using OUTPUT parameter stmt	returns value using return
4 allows dmls	doesn't allow dml stmts
5 cannot be executed from sql commands	can be executed from sql commands
6 created to perform actions like insert,update,	created for calculations

delete on tables

7 create procedure to update balance create function to
get balance

=> difference between scalar and table valued functions ?

	SCALAR	TABLE
1	returns one value	returns rows
2	return type must be scalar types	return type must be TABLE
3	return expr must be a scalar variable	return expr must be select query
4	called in select clause	called in from clause

=> list of procedures & functions created by user ?

SELECT * FROM INFORMATION_SCHEMA.ROUTINES

Dropping :-

=====

DROP PROCEDURE raise_salary
DROP FUNCTION getExpr

=====

TRIGGERS :-

=====

=> a trigger is also a named T-SQL block like procedure but executed implicitly by sql server

whenever user executes DML / DDL commands

=> triggers are created

- 1 to control dml operations
- 2 to enforce complex rules and validations
- 3 to audit day-to-day operations on tables

CREATE OR ALTER TRIGGER <name>
ON <TABNAME>
AFTER / INSTEAD OF INSERT,UPDATE,DELETE
AS
 statements

AFTER triggers :-

=====

=> if trigger is after then sql server executes the trigger after executing dml

INSTEAD OF triggers :-

=====

=> if trigger is instead of then sql server executes the trigger instead of executing dml

Ex :-

=> create trigger to not to allow dmls on emp table on sunday ?

```
CREATE OR ALTER TRIGGER TRG1
ON EMP
AFTER INSERT,UPDATE,DELETE
AS
    IF DATENAME(DW,GETDATE()) = 'SUNDAY'
    BEGIN
        ROLLBACK
        RAISERROR('sunday not allowed',16,1)
    END
```

=> create trigger to not to allow dmls on emp table as follows ?

```
MON - FRI    <10AM AND >4PM
SAT          <10AM AND >2PM
SUN          -----
```

=> create trigger to not to allow to update empno ?

```
CREATE OR ALTER TRIGGER TRG3
ON EMP
AFTER UPDATE
AS
    IF UPDATE(empno) OR UPDATE(hiredate)
    BEGIN
        ROLLBACK
        RAISERROR('cannot update empno and hiredate',16,1)
    END
```

29-may-26

Magic Tables :-
=====

- 1 INSERTED
- 2 DELETED

=> These tables are called magic tables because these tables are created automatically and destroyed automatically.

=> these tables can be accessed only with in trigger

=> record user is trying to insert is copied to inserted table

=> record user is trying to delete is copied to deleted table

INSERT INTO emp VALUES(100, 'A', 'CLERK', -----4000, 200, 20) => INSERTED

DELETE FROM emp WHERE empno = 107 => DELETED

UPDATE emp SET sal = 10000 WHERE empno = 108 => INSETED

empno sal

108 10000

DELETED
empno sal
108 6000

Ex 4 :-

=> create a trigger to not to allow to decrement salary ?

```
CREATE OR ALTER TRIGGER TRG4
ON EMP
AFTER UPDATE
AS
    DECLARE @oldsal MONEY,@newsal MONEY
    SELECT @oldsal = sal FROM DELETED
    SELECT @newsal = sal FROM INSERTED
    IF @newsal < @oldsal
    BEGIN
        ROLLBACK
        RAISERROR('sal cannot be decremented',16,1)
    END
```

Testing :-

UPDATE emp SET sal=3000 WHERE empno = 108 => ERROR

Ex 5 :-

=> create trigger to insert details into emp_resign whenever employee resigns ?

```
EMP_RESIGN
empno      ename      job      sal      hiredate  dor

CREATE TABLE emp_resign
(
    empno INT,
    ename VARCHAR(10),
    job VARCHAR(10),
    sal MONEY,
    hiredate DATE,
    dor DATE
)

CREATE OR ALTER TRIGGER TRG5
ON emp
AFTER DELETE
AS
    INSERT INTO emp_resign
    SELECT empno,ename,job,sal,hiredate,getdate() FROM DELETED
```

Testing :-

DELETE FROM emp WHERE empno = 100

```
/* 1 row is copied to deleted table
   2 delete command is executed
   3 trigger is executed
*/
```

Ex 6 :-

```
CUSTS                                CUSTT
CID  NAME  CITY                      CID  NAME  CITY

CREATE OR ALTER TRIGGER TRG6
ON CUSTS
```

```

AFTER INSERT,UPDATE,DELETE
AS
    DECLARE @cnt1 INT,@cnt2 INT
    DECLARE @ncid INT,@nname VARCHAR(10),@ncity VARCHAR(10)
    DECLARE @ocid INT,@oname VARCHAR(10),@ocity VARCHAR(10)
    SELECT @cnt1 = COUNT(*) FROM INSERTED
    SELECT @cnt2 = COUNT(*) FROM DELETED
    SELECT @ncid = cid,@nname = cname,@ncity = city FROM INSERTED
    SELECT @ocid = cid,@oname = cname,@ocity = city FROM DELETED
    IF @cnt1 =1 AND @cnt2=0
        INSERT INTO custt VALUES(@ncid,@nname,@ncity)
    ELSE IF @cnt1=0 AND @cnt2=1
        DELETE FROM custt WHERE cid = @ocid
    ELSE
        UPDATE custt SET city = @ocity WHERE cid = @ocid

```

Testing :-

```

INSERT INTO CUSTS VALUES(100,'A','HYD')

SELECT * FROM CUSTT

UPDATE CUSTS SET CITY='BLR' WHERE CID=100

SELECT * FROM CUSTT

DELETE FROM CUSTS WHERE CID = 100

SELECT * FROM CUSTT

```

30-may-26

INSTEAD OF triggers :-
=====

=> if trigger is instead of the sql server executes the trigger instead of executing dml

ex :-

=> create trigger to not to allow more than 4 employees per dept ?

```

EMP22
ENO      ENAME DNO
1         10
2         10
3         10
4         10
5         10 => not allowed

```

```

CREATE OR ALTER TRIGGER TRG7
ON EMP22
INSTEAD OF INSERT
AS
    DECLARE @eno INT,@name VARCHAR(10),@dno INT,@cnt INT
    SELECT @eno = eno,@name=enname,@dno = dno FROM INSERTED
    SELECT @cnt = COUNT(*) FROM emp22 WHERE dno = @dno
    IF @cnt >= 4
        RAISERROR('max 4 emps per dept',16,1)
    ELSE
        INSERT INTO EMP22 VALUES(@eno,@name,@dno)

```

Testing :-

```
INSERT INTO EMP22 VALUES(1, 'A', 10)
INSERT INTO EMP22 VALUES(2, 'B', 10)
INSERT INTO EMP22 VALUES(3, 'C', 10)
INSERT INTO EMP22 VALUES(4, 'D', 10)
INSERT INTO EMP22 VALUES(5, 'E', 10) => ERROR
```

=> list of triggers ?

```
SELECT * FROM SYS.TRIGGERS
```

=> display trigger names and table names ?

SYS.TRIGGERS			SYS.TABLES		
	name	object_id	parent_id	name	object_id
	SELECT tr.name as trig , tb.name as tab				
	FROM sys.triggers as tr INNER JOIN sys.tables as tb				
	ON tr.parent_id = tb.object_id				
	TRG1	EMP			
TRG3	EMP				
TRG4	EMP				
TRG5	EMP				
TRG6	CUSTS				
TRG7	EMP22				

Dropping :-

=====

```
DROP TRIGGER TRG1
```

if we drop table what about triggers created on table ?

ANS :- triggers are also dropped

SERVER	DATABASE	TABLES	ROWS & COLS
			CONSTRAINTS
			INDEXES
			TRIGGERS
		VIEWS	
		SYNONYMS	
		SEQUENCES	
		PROCEDURES	
		FUNCTIONS	

=====

Dynamic SQL :-

=====

=> "sql" command generated at runtime are called dynamic sql commands

ex :- DROP TABLE emp (static sql)

```
DECLARE @tname VARCHAR(20)
SET @tname = 'EMP'
```

DROP TABLE @tname (dynamic sql)

=> Dynamic sql is useful when we don't know table names and column names until runtime

=> Dynamic sql command are executed by using EXEC stmt

=> Dynamic sql command that you want to execute should be passed as a string to EXEC

```
EXEC ( ' dynamic sql command ' )
```

Ex 1 :-

```
DECLARE @tname VARCHAR(20)
SET @tname = 'EMP22'
EXEC ( ' DROP TABLE ' + @tname)
```

Ex 2 :-

=> create a procedure to drop all tables from db ?

```
CREATE OR ALTER PROCEDURE DROP_ALL_TABLES
AS
    DECLARE C1 CURSOR FOR SELECT TABLE_NAME
                                FROM
INFORMATION_SCHEMA.TABLES
                                WHERE TABLE_TYPE='BASE
TABLE'
    DECLARE @tname VARCHAR(30)
    OPEN C1
    FETCH NEXT FROM C1 INTO @tname
    WHILE(@@FETCH_STATUS=0)
    BEGIN
        EXEC ( ' DROP TABLE ' + @tname)
        FETCH NEXT FROM C1 INTO @tname
    END
    CLOSE C1
    DEALLOCATE C1
```

EXECUTE :-

```
EXECUTE DROP_ALL_TABLES
```

1-jun-26

Normalization :-

=====

=> Normalization is the process of decomposing tables with redundancy into number of well structured tables

Advantages :-

=====

1 Eliminates Data Redundancy :-

=> Divides large, repeating tables into smaller tables ensuring each piece of information is stored in only one place

2 Improves Data Consistency :-

=> Because data is stored only once, any update operation only requires a single change, preventing mismatched or conflicting records across the database.

3 Prevents Data Anomalies :-

=> Structured rules (like 1NF, 2NF, 3NF) prevent errors and accidental data loss during record insertion, modification, or deletion

4 Enforces Data Integrity:

=> Organizes data using strict primary and foreign keys, protecting the logical relationships between different tables.

5 Easier Maintenance:

=> Smaller, specialized tables make schema modifications and long-term database maintenance easier and less risky.

=> Normalization is set of rules and each rule is called one normal form
=> There are six normal forms

1NF
2NF
3NF
BCNF (boyce-codd NF)
4NF
5NF

Ex :-

BILL										
billno	bdate	ccode	cname	addr	icode	name	rate	qty	value	tbill
1000	?	100	A		HYD	10	AAA	100	2	200
200										
1000	?	100	A		HYD	11	BBB	50	2	150
350										
			12	CCC	150	2	300		650	
			20							

=> above table contains redundancy , to reduce redundancy apply normalization process on above table

1NF :-
=====

=> a table said to be in 1NF if there are no multivalued attributes in it or all the attributes in table are atomic (single)

BILL										
billno	bdate	ccode	cname	addr	icode	name	rate	qty	value	tbill
s	s	s	s	s		s	m		m	m
m	m	s								

=> above table contains multivalued attributes ,so the table is not according to

1NF then decompose the table into two tables and establish relationship by adding foreign key.

BILL					
billno	bdate	ccode	cname	addr	tbill

1000	?	100	A	HYD	3000

1001	?	101	B	HYD	4000
------	---	-----	---	-----	------

ITEMS					
billno	icode	name	rate	qty	value

1000	10	AAA	100	2	200
1000	11	BBB			
1000	12	CCC			
1001	10	AAA	100	3	300

=> In the above table if icode is repated then name,rate are also repeated so the above table contain redundancy , so to reduce redundancy apply 2NF

2NF :-
=====

=> a table said to be in 2NF

- 1 if it is in 1NF
- 2 if there are no partial dependencies in it

R(A,B,C,D) A => primary key
A =====> B,C,D (full dependency)

R(A,B,C,D) A,B => primary key
A,B =====> C (full dependency)
B =====> D (partial dependency)

TABLE 1 :-

BILL						
billno	bdate	ccode	cname	addr	tbill	

1000	?	100	A	HYD	3000	
1001	?	101	B	HYD	4000	

=> above table satisfies 2NF because there are no partial dependencies exists

TABLE 2:-

ITEMS					
billno	icode	name	rate	qty	value

1000	10	AAA	100	2	200
1000	11	BBB			
1000	12	CCC			
1001	10	AAA	100	3	300

billno,icode =====> qty , value (full dependency)
icode =====> name,rate (partial dependency)

=> above table contains partial dependency which is not allowed according to 2NF
then decompose the table as follows

TABLE 2 :-


```

ITEMS
icode    name    rate
-----

```

TABLE 3 :-

```

BILL_ITEMS
billno   icode    qty    value
-----

```

3NF :-
=====

=> a table said to be in 3NF

- 1 if it is in 2NF
- 2 if there are no transitive dependencies in it
- 3 if table contains any derived attributes then we can permanently remove them

R(A,B,C,D) A => pk

A =====>B,C (full dependency)

C =====> D (transitive dependency)

TABLE 1 :-

```

BILL
billno   bdate    ccode   cname   addr    tbill
-----
1000     ?         100     A       HYD     3000
1001     ?         101     B       HYD     4000

```

billno =====> bdate , tbill (full dependency)
ccode =====> cname,addr (transitive dependency)

=> above table contains transitive dependency , so the table is not according to 3NF

then decompose the table as follows

```

CUST
ccode    cname addr
-----

```

```

BILL
billno   bdate tbill (derived)   ccode (fk)
-----

```

TABLE 2 :-

```

ITEMS
icode    name    rate
-----

```

TABLE 3 :-

```

BILL_ITEMS
billno   icode    qty    value (derived)
-----

```

AFTER 3NF :-
=====

CUST
ccode cname addr

BILL
billno bdate ccode (fk)

ITEMS
icode name rate

BILL_ITEMS
billno icode qty

TABLE LEVEL :-
=====

